

ANFIS: Adaptive-Network-Based Fuzzy Inference System

Jyh-Shing Roger Jang

Abstract—The architecture and learning procedure underlying ANFIS (adaptive-network-based fuzzy inference system) is presented, which is a fuzzy inference system implemented in the framework of adaptive networks. By using a hybrid learning procedure, the proposed ANFIS can construct an input–output mapping based on both human knowledge (in the form of fuzzy if–then rules) and stipulated input–output data pairs. In the simulation, the ANFIS architecture is employed to model nonlinear functions, identify nonlinear components on-line in a control system, and predict a chaotic time series, all yielding remarkable results. Comparisons with artificial neural networks and earlier work on fuzzy modeling are listed and discussed. Other extensions of the proposed ANFIS and promising applications to automatic control and signal processing are also suggested.

I. INTRODUCTION

SYSTEM MODELING based on conventional mathematical tools (e.g., differential equations) is not well suited for dealing with ill-defined and uncertain systems. By contrast, a fuzzy inference system employing fuzzy if–then rules can model the qualitative aspects of human knowledge and reasoning processes without employing precise quantitative analyses. This *fuzzy modeling* or *fuzzy identification*, first explored systematically by Takagi and Sugeno [54], has found numerous practical applications in control [36], [46], prediction and inference [16], [17]. However, there are some basic aspects of this approach which are in need of better understanding. More specifically:

- 1) No standard methods exist for transforming human knowledge or experience into the rule base and database of a fuzzy inference system.
- 2) There is a need for effective methods for tuning the membership functions (MF's) so as to minimize the output error measure or maximize performance index.

In this perspective, the aim of this paper is to suggest a novel architecture called *Adaptive-Network-based Fuzzy Inference System*, or simply *ANFIS*, which can serve as a basis for constructing a set of fuzzy if–then rules with appropriate membership functions to generate the stipulated input–output pairs. The next section introduces the basics of fuzzy if–then rules and fuzzy inference systems. Section III describes the structures and learning rules of adaptive networks. By embedding the fuzzy inference system into the framework of

adaptive networks, we obtain the ANFIS architecture which is the backbone of this paper and it is covered in Section IV. Application examples such as nonlinear function modeling and chaotic time series prediction are given in Section V. Section VI concludes this paper by giving important extensions and future directions of this work.

II. FUZZY IF-THEN RULES AND FUZZY INFERENCE SYSTEMS

A. Fuzzy If-Then Rules

Fuzzy if-then rules or *fuzzy conditional statements* are expressions of the form *IF A THEN B*, where *A* and *B* are labels of *fuzzy sets* [66] characterized by appropriate membership functions. Due to their concise form, fuzzy if–then rules are often employed to capture the imprecise modes of reasoning that play an essential role in the human ability to make decisions in an environment of uncertainty and imprecision. An example that describes a simple fact is

If pressure is high, then volume is small

where *pressure* and *volume* are *linguistic variables* [67], *high* and *small* are *linguistic values* or *labels* that are characterized by membership functions.

Another form of fuzzy if–then rule, proposed by Takagi and Sugeno [53], has fuzzy sets involved only in the premise part. By using Takagi and Sugeno's fuzzy if–then rule, we can describe the resistant force on a moving object as follows:

*If velocity is high, then force = $k * (\text{velocity})^2$*

where, again, *high* in the premise part is a linguistic label characterized by an appropriate membership function. However, the consequent part is described by a nonfuzzy equation of the input variable, velocity.

Both types of fuzzy if–then rules have been used extensively in both modeling and control. Through the use of linguistic labels and membership functions, a fuzzy if–then rule can easily capture the spirit of a “rule of thumb” used by humans. From another angle, due to the qualifiers on the premise parts, each fuzzy if–then rule can be viewed as a local description of the system under consideration. Fuzzy if–then rules form a core part of the fuzzy inference system to be introduced below.

A. Fuzzy Inference Systems

Fuzzy inference systems are also known as *fuzzy-rule-based systems*, *fuzzy models*, *fuzzy associative memories (FAM)*, or *fuzzy controllers* when used as controllers. Basically a fuzzy

Manuscript received July 30, 1991; revised October 27, 1992. This work was supported in part by NASA Grant NCC 2-275, in part by MICRO Grant 92-180, and in part by EPRI Agreement RP 8010-34.

The author is with the Department of Electrical Engineering and Computer Science, University of California, Berkeley, CA 94720
IEEE Log Number 9207521.

0018-9472/93\$03.00 © 1993 IEEE

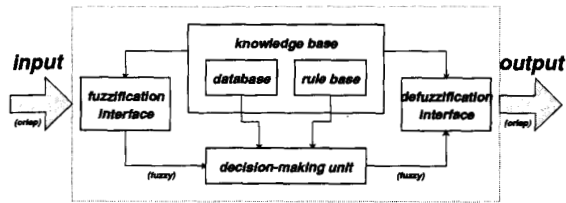


Fig. 1. Fuzzy inference system.

inference system is composed of five functional blocks (see Fig. 1):

- a *rule base* containing a number of fuzzy if-then rules;
- a *database* which defines the membership functions of the fuzzy sets used in the fuzzy rules;
- a *decision-making unit* which performs the inference operations on the rules;
- a *fuzzification interface* which transforms the crisp inputs into degrees of match with linguistic values;
- a *defuzzification interface* which transform the fuzzy results of the inference into a crisp output.

Usually, the rule base and the database are jointly referred to as the *knowledge base*.

The steps of *fuzzy reasoning* (inference operations upon fuzzy if-then rules) performed by fuzzy inference systems are:

- 1) Compare the input variables with the membership functions on the premise part to obtain the membership values (or compatibility measures) of each linguistic label. (This step is often called *fuzzification*).
- 2) Combine (through a specific T-norm operator, usually multiplication or min.) the membership values on the premise part to get *firing strength (weight)* of each rule.
- 3) Generate the qualified consequent (either fuzzy or crisp) of each rule depending on the firing strength.
- 4) Aggregate the qualified consequents to produce a crisp output. (This step is called *defuzzification*.)

Several types of fuzzy reasoning [23], [24] have been proposed in the literature. Depending on the types of fuzzy reasoning and fuzzy if-then rules employed, most fuzzy inference systems can be classified into three types (see Fig. 2):

- Type 1:* The overall output is the weighted average of each rule's crisp output induced by the rule's firing strength (the product or minimum of the degrees of match with the premise part) and output membership functions. The output membership functions used in this scheme must be monotonic functions [55].
- Type 2:* The overall fuzzy output is derived by applying "max" operation to the qualified fuzzy outputs (each of which is equal to the minimum of firing strength and the output membership function of each rule). Various schemes have been proposed to choose the final crisp output based on the overall fuzzy output; some of them are centroid of area, bisector of area, mean of maxima, maximum criterion, etc [23], [24].
- Type 3:* Takagi and Sugeno's fuzzy if-then rules are used [53]. The output of each rule is a linear combination of

input variables plus a constant term, and the final output is the weighted average of each rule's output.

Fig. 2 utilizes a two-rule two-input fuzzy inference system to show different types of fuzzy rules and fuzzy reasoning mentioned above. Be aware that most of the differences come from the specification of the consequent part (monotonically non-decreasing or bell-shaped membership functions, or crisp function) and thus the defuzzification schemes (weighted average, centroid of area, etc) are also different.

III. ADAPTIVE NETWORKS: ARCHITECTURES AND LEARNING ALGORITHMS

This section introduces the architecture and learning procedure of the adaptive network which is in fact a superset of all kinds of feedforward neural networks with supervised learning capability. An adaptive network, as its name implies, is a network structure consisting of nodes and directional links through which the nodes are connected. Moreover, part or all of the nodes are adaptive, which means their outputs depend on the parameter(s) pertaining to these nodes, and the learning rule specifies how these parameters should be changed to minimize a prescribed error measure.

The basic learning rule of adaptive networks is based on the gradient descent and the chain rule, which was proposed by Werbos [61] in the 1970's. However, due to the state of artificial neural network research at that time, Werbos' early work failed to receive the attention it deserved. In the following presentation, the derivation is based on the author's work [11], [10] which generalizes the formulas in [39].

Since the basic learning rule is based the gradient method which is notorious for its slowness and tendency to become trapped in local minima, here we propose a hybrid learning rule which can speed up the learning process substantially. Both the batch learning and the pattern learning of the proposed hybrid learning rule discussed below.

A. Architecture and Basic Learning Rule

An adaptive network (see Fig. 3) is a multilayer feedforward network in which each node performs a particular function (*node function*) on incoming signals as well as a set of parameters pertaining to this node. The formulas for the node functions may vary from node to node, and the choice of each node function depends on the overall input-output function which the adaptive network is required to carry out. Note that the links in an adaptive network only indicate the flow direction of signals between nodes; no weights are associated with the links.

To reflect different adaptive capabilities, we use both circle and square nodes in an adaptive network. A square node (adaptive node) has parameters while a circle node (fixed node) has none. The parameter set of an adaptive network is the union of the parameter sets of each adaptive node. In order to achieve a desired input-output mapping, these parameters are updated according to given training data and a gradient-based learning procedure described below.

Suppose that a given adaptive network has L layers and the k th layer has $\#(k)$ nodes. We can denote the node in the

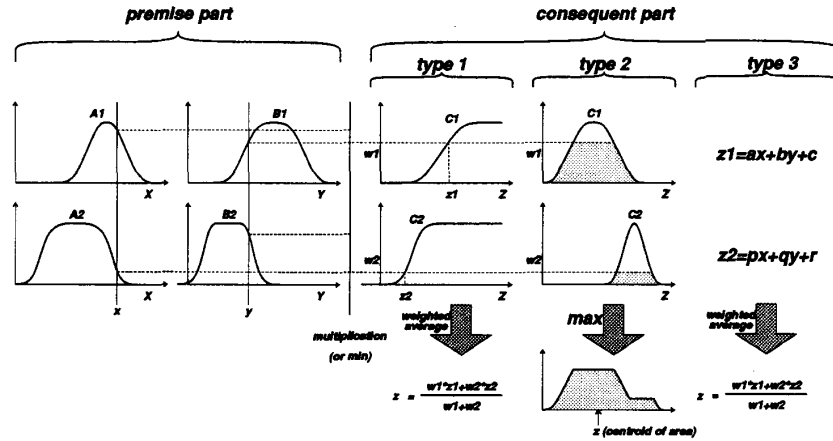


Fig. 2. Commonly used fuzzy if-then rules and fuzzy reasoning mechanisms.

i th position of the k th layer by (k, i) , and its node function (or node output) by O_i^k . Since a node output depends on its incoming signals and its parameter set, we have

$$O_i^k = O_i^k(O_1^{k-1}, \dots, O_{\#(k-1)}^{k-1}, a, b, c, \dots) \quad (1)$$

where a, b, c , etc., are the parameters pertaining to this node. (Note that we use O_i^k as both the node output and node function.)

Assuming the given training data set has P entries, we can define the *error measure* (or *energy function*) for the p th ($1 \leq p \leq P$) entry of training data entry as the sum of squared errors:

$$E_p = \sum_{m=1}^{\#(L)} (T_{m,p} - O_{m,p}^L)^2 \quad (2)$$

where $T_{m,p}$ is the m th component of p th target output vector, and $O_{m,p}^L$ is the m th component of actual output vector produced by the presentation of the p th input vector. Hence the overall error measure is $E = \sum_{p=1}^P E_p$.

In order to develop a learning procedure that implements gradient descent in E over the parameter space, first we have to calculate the *error rate* $\partial E_p / \partial O$ for p th training data and for each node output O . The error rate for the output node at (L, i) can be calculated readily from (2):

$$\frac{\partial E_p}{\partial O_{i,p}^L} = -2(T_{i,p} - O_{i,p}^L). \quad (3)$$

For the internal node at (k, i) , the error rate can be derived by the chain rule:

$$\frac{\partial E_p}{\partial O_{i,p}^k} = \sum_{m=1}^{\#(k+1)} \frac{\partial E_p}{\partial O_{m,p}^{k+1}} \frac{\partial O_{m,p}^{k+1}}{\partial O_{i,p}^k} \quad (4)$$

where $1 \leq k \leq L-1$. That is, the error rate of an internal node can be expressed as a linear combination of the error rates of the nodes in the next layer. Therefore for all $1 \leq k \leq L$ and $1 \leq i \leq \#(k)$, we can find $\partial E_p / \partial O_{i,p}^k$ by (3) and (4).

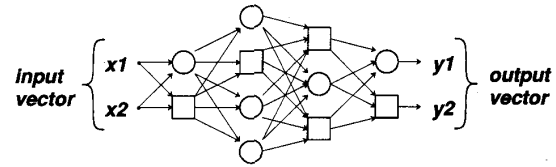


Fig. 3. An adaptive network.

Now if α is a parameter of the given adaptive network, we have

$$\frac{\partial E_p}{\partial \alpha} = \sum_{O^* \in S} \frac{\partial E_p}{\partial O^*} \frac{\partial O^*}{\partial \alpha}, \quad (5)$$

where S is the set of nodes whose outputs depend on α . Then the derivative of the overall error measure E with respect to α is

$$\frac{\partial E}{\partial \alpha} = \sum_{p=1}^P \frac{\partial E_p}{\partial \alpha}. \quad (6)$$

Accordingly, the update formula for the generic parameter α is

$$\Delta \alpha = -\eta \frac{\partial E}{\partial \alpha} \quad (7)$$

in which η is a learning rate which can be further expressed as

$$\eta = \frac{k}{\sqrt{\sum_{\alpha} \left(\frac{\partial E}{\partial \alpha} \right)^2}} \quad (8)$$

where k is the *step size*, the length of each gradient transition in the parameter space. Usually, we can change the value of k to vary the speed of convergence. The heuristic rules for changing k are discussed in the Section V where we report simulation results.

Actually, there are two learning paradigms for adaptive networks. With the *batch learning* (or *off-line learning*), the update formula for parameter α is based on (6) and the update action takes place only after the whole training data set has been presented, i.e., only after each *epoch* or *sweep*.

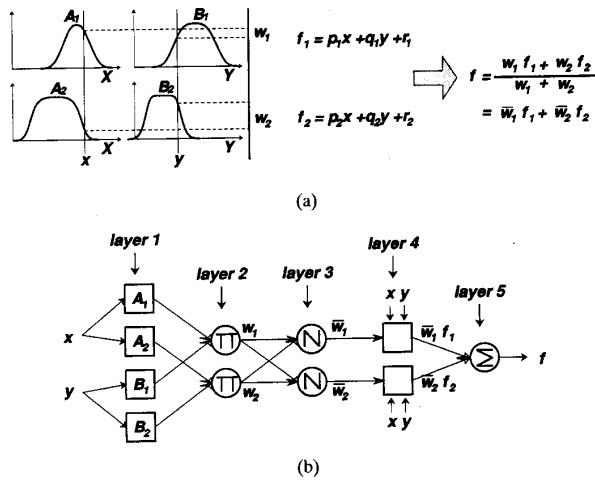


Fig. 4. (a) Type-3 fuzzy reasoning. (b) Equivalent ANFIS (type-3 ANFIS).

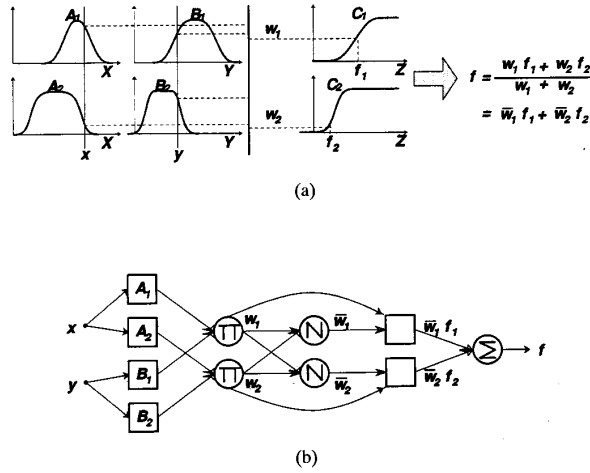


Fig. 5. (a) Type-1 fuzzy reasoning. (b) Equivalent ANFIS (type-1 ANFIS).

On the other hand, if we want the parameters to be updated immediately after each input-output pair has been presented, then the update formula is based on (5) and it is referred to as the *pattern learning* (or *on-line learning*). In the following we will derive a faster hybrid learning rule and both of its learning paradigms.

B. Hybrid Learning Rule: Batch (Off-Line) Learning

Though we can apply the gradient method to identify the parameters in an adaptive network, the method is generally slow and likely to become trapped in local minima. Here we propose a hybrid learning rule [10] which combines the gradient method and the least squares estimate (LSE) to identify parameters.

For simplicity, assume that the adaptive network under consideration has only one output

$$\text{output} = F(\vec{I}, S) \quad (9)$$

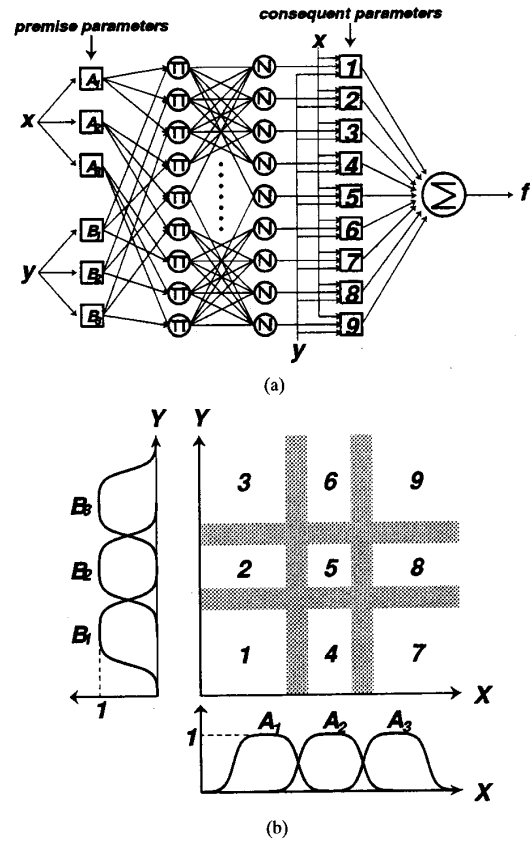


Fig. 6. (a) Two-input type-3 ANFIS with nine rules. (b) Corresponding fuzzy subspaces.

where $\vec{I}$ is the set of input variables and S is the set of parameters. If there exists a function H such that the composite function $H \circ F$ is linear in some of the elements of S , then these elements can be identified by the least squares method. More formally, if the parameter set S can be decomposed into two sets

$$S = S_1 \oplus S_2 \quad (10)$$

(where $\oplus$ represents direct sum) such that $H \circ F$ is linear in the elements of S_2 , then upon applying H to (9), we have

$$H(\text{output}) = H \circ F(\vec{I}, S) \quad (11)$$

which is linear in the elements of S_2 . Now given values of elements of S_1 , we can plug P training data into (11) and obtain a matrix equation:

$$AX = B \quad (12)$$

where X is an unknown vector whose elements are parameters in S_2 . Let $|S_2| = M$, then the dimensions of A , X and B are $P \times M$, $M \times 1$ and $P \times 1$, respectively. Since P (number of training data pairs) is usually greater than M (number of linear parameters), this is an overdetermined problem and generally there is no exact solution to (12). Instead, a *least squares estimate (LSE)* of X , X^* , is sought to minimize the

squared error $\|AX - B\|^2$. This is a standard problem that forms the grounds for linear regression, adaptive filtering and signal processing. The most well-known formula for X^* uses the pseudo-inverse of X :

$$X^* = (A^T A)^{-1} A^T B \quad (13)$$

where A^T is the transpose of A , and $(A^T A)^{-1} A^T$ is the pseudo-inverse of A if $A^T A$ is non-singular. While (13) is concise in notation, it is expensive in computation when dealing with the matrix inverse and, moreover, it becomes ill-defined if $A^T A$ is singular. As a result, we employ sequential formulas to compute the LSE of X . This sequential method of LSE is more efficient (especially when M is small) and can be easily modified to an on-line version (see below) for systems with changing characteristics. Specifically, let the i th row vector of matrix A defined in (12) be a_i^T and the i th element of B be b_i^T , then X can be calculated iteratively using the sequential formulas widely adopted in the literature [1], [7], [26], [45]:

$$\left. \begin{aligned} X_{i+1} &= X_i + S_{i+1} a_{i+1} (b_{i+1}^T - a_{i+1}^T X_i) \\ S_{i+1} &= S_i - \frac{S_i a_{i+1} a_{i+1}^T S_i}{1 + a_{i+1}^T S_i a_{i+1}}, \quad i = 0, 1, \dots, P-1 \end{aligned} \right\} \quad (14)$$

where S_i is often called the *covariance matrix* and the least squares estimate X^* is equal to X_P . The initial conditions to bootstrap (14) are $X_0 = 0$ and $S_0 = \gamma I$, where γ is a positive large number and I is the identity matrix of dimension $M \times M$. When dealing with multi-output adaptive networks (*output* in (9) is a column vector), (14) still applies except that b_i^T is the i th rows of matrix B .

Now we can combine the gradient method and the least squares estimate to update the parameters in an adaptive network. Each epoch of this hybrid learning procedure is composed of a forward pass and a backward pass. In the forward pass, we supply input data and functional signals go forward to calculate each node output until the matrices A and B in (12) are obtained, and the parameters in S_2 are identified by the sequential least squares formulas in (14). After identifying parameters in S_2 , the functional signals keep going forward till the error measure is calculated. In the backward pass, the error rates (the derivative of the error measure w.r.t. each node output, see (3) and (4)) propagate from the output end toward the input end, and the parameters in S_1 are updated by the gradient method in (7).

For given fixed values of parameters in S_1 , the parameters in S_2 thus found are guaranteed to be the global optimum point in the S_2 parameter space due to the choice of the squared error measure. Not only can this hybrid learning rule decrease the dimension of the search space in the gradient method, but, in general, it will also cut down substantially the convergence time.

Take for example an one-hidden-layer back-propagation neural network with sigmoid activation functions. If this neural network has p output units, then the *output* in (9) is a column vector. Let $H(\cdot)$ be the inverse sigmoid function

$$H(x) = \ln\left(\frac{x}{1-x}\right) \quad (15)$$

TABLE I
TWO PASSES IN THE HYBRID LEARNING PROCEDURE FOR ANFIS

	Forward Pass	Backward Pass
Premise Parameters	Fixed	Gradient Descent
Consequent Parameters	Least Squares Estimate	Fixed
Signals	Node Outputs	Error Rates

then (11) becomes a linear (vector) function such that each element of $H(\text{output})$ is a linear combination of the parameters (weights and thresholds) pertaining to layer 2. In other words, S_1 = weights and thresholds of hidden layer
 S_2 = weights and thresholds of output layer.

Therefore we can apply the back-propagation learning rule to tune the parameters in the hidden layer, and the parameters in the output layer can be identified by the least squares method. However, it should be keep in mind that by using the least squares method on the data transformed by $H(\cdot)$, the obtained parameters are optimal in terms of the transformed squared error measure instead of the original one. Usually this will not cause practical problem as long as $H(\cdot)$ is monotonically increasing.

C. Hybrid Learning Rule: Pattern (On-Line) Learning

If the parameters are updated after each data presentation, we have the *pattern learning* or *on-line learning* paradigm. This learning paradigm is vital to the on-line parameter identification for systems with changing characteristics. To modify the batch learning rule to its on-line version, it is obvious that the gradient descent should be based on E_p (see (5)) instead of E . Strictly speaking, this is not a truly gradient search procedure to minimize E , yet it will approximate to one if the learning rate is small.

For the sequential least squares formulas to account for the time-varying characteristics of the incoming data, we need to decay the effects of old data pairs as new data pairs become available. Again, this problem is well studied in the adaptive control and system identification literature and a number of solutions are available [7]. One simple method is to formulate the squared error measure as a weighted version that gives higher weighting factors to more recent data pairs. This amounts to the addition of a *forgetting factor* λ to the original sequential formula:

$$\left. \begin{aligned} X_{i+1} &= X_i + S_{i+1} a_{i+1} (b_{i+1}^T - a_{i+1}^T X_i) \\ S_{i+1} &= \frac{1}{\lambda} \left[S_i - \frac{S_i a_{i+1} a_{i+1}^T S_i}{\lambda + a_{i+1}^T S_i a_{i+1}} \right] \end{aligned} \right\}, \quad (16)$$

where the value of λ is between 0 and 1. The smaller λ is, the faster the effects of old data decay. But a small λ sometimes causes numerical instability and should be avoided.

IV. ANFIS: ADAPTIVE-NETWORK-BASED FUZZY INFERENCE SYSTEM

The architecture and learning rules of adaptive networks have been described in the previous section. Functionally, there are almost no constraints on the node functions of an adaptive network except piecewise differentiability. Structurally, the only limitation of network configuration is that

it should be of feedforward type. Due to these minimal restrictions, the adaptive network's applications are immediate and immense in various areas. In this section, we propose a class of adaptive networks which are functionally equivalent to fuzzy inference systems. The proposed architecture is referred to as ANFIS, standing for adaptive-network-based fuzzy inference system. We describe how to decompose the parameter set in order to apply the hybrid learning rule. Besides, we demonstrate how to apply the Stone-Weierstrass theorem to ANFIS with simplified fuzzy if-then rules and how the radial basis function network relate to this kind of simplified ANFIS.

A. ANFIS Architecture

For simplicity, we assume the fuzzy inference system under consideration has two inputs x and y and one output z . Suppose that the rule base contains two fuzzy if-then rules of Takagi and Sugeno's type [53].

Rule 1: If x is A_1 and y is B_1 , then $f_1 = p_1x + q_1y + r_1$,
Rule 2: If x is A_2 and y is B_2 , then $f_2 = p_2x + q_2y + r_2$.

Then the type-3 fuzzy reasoning is illustrated in Fig. 4(a), and the corresponding equivalent ANFIS architecture (type-3 ANFIS) is shown in Fig. 4(b). The node functions in the same layer are of the same function family as described below:

Layer 1: Every node i in this layer is a square node with a node function

$$O_i^1 = \mu_{A_i}(x) \quad (17)$$

where x is the input to node i , and A_i is the linguistic label (*small*, *large*, etc.) associated with this node function. In other words, O_i^1 is the membership function of A_i and it specifies the degree to which the given x satisfies the quantifier A_i . Usually we choose $\mu_{A_i}(x)$ to be bell-shaped with maximum equal to 1 and minimum equal to 0, such as

$$\mu_{A_i}(x) = \frac{1}{1 + \left[\left(\frac{x-c_i}{a_i}\right)^2\right]^{b_i}} \quad (18)$$

or

$$\mu_{A_i}(x) = \exp\left\{-\left(\frac{x-c_i}{a_i}\right)^2\right\} \quad (19)$$

where $\{a_i, b_i, c_i\}$ is the parameter set. As the values of these parameters change, the bell-shaped functions vary accordingly, thus exhibiting various forms of membership functions on linguistic label A_i . In fact, any continuous and piecewise differentiable functions, such as commonly used trapezoidal or triangular-shaped membership functions, are also qualified candidates for node functions in this layer. Parameters in this layer are referred to as *premise parameters*.

Layer 2: Every node in this layer is a circle node labeled Π which multiplies the incoming signals and sends the product out. For instance,

$$w_i = \mu_{A_i}(x) \times \mu_{B_i}(y), \quad i = 1, 2. \quad (20)$$

Each node output represents the firing strength of a rule. (In fact, other T -norm operators that perform generalized AND can be used as the node function in this layer.)

Layer 3: Every node in this layer is a circle node labeled N . The i th node calculates the ratio of the i th rule's firing strength to the sum of all rules' firing strengths:

$$\bar{w}_i = \frac{w_i}{w_1 + w_2}, \quad i = 1, 2. \quad (21)$$

For convenience, outputs of this layer will be called *normalized firing strengths*.

Layer 4: Every node i in this layer is a square node with a node function

$$O_i^4 = \bar{w}_i f_i = \bar{w}_i(p_i x + q_i y + r_i) \quad (22)$$

where $\bar{w}_i$ is the output of layer 3, and $\{p_i, q_i, r_i\}$ is the parameter set. Parameters in this layer will be referred to as *consequent parameters*.

Layer 5: The single node in this layer is a circle node labeled Σ that computes the overall output as the summation of all incoming signals, i.e.,

$$O_1^5 = \text{overall output} = \sum_i \bar{w}_i f_i = \frac{\sum_i w_i f_i}{\sum_i w_i}. \quad (23)$$

Thus we have constructed an adaptive network which is functionally equivalent to a type-3 fuzzy inference system. For type-1 fuzzy inference systems, the extension is quite straightforward and the type-1 ANFIS is shown in Fig. 5 where the output of each rule is induced jointly by the output membership function and the firing strength. For type-2 fuzzy inference systems, if we replace the centroid defuzzification operator with a discrete version which calculates the approximate centroid of area, then type-3 ANFIS can still be constructed accordingly. However, it will be more complicated than its type-3 and type-1 versions and thus not worth the efforts to do so.

Fig. 6 shows a 2-input, type-3 ANFIS with nine rules. Three membership functions are associated with each input, so the input space is partitioned into nine fuzzy subspaces, each of which is governed by a fuzzy if-then rules. The premise part of a rule delineates a fuzzy subspace, while the consequent part specifies the output within this fuzzy subspace.

B. Hybrid Learning Algorithm

From the proposed type-3 ANFIS architecture (see Fig. 4), it is observed that given the values of premise parameters, the overall output can be expressed as a linear combinations of the consequent parameters. More precisely, the output f in Fig. 4 can be rewritten as

$$\begin{aligned} f &= \frac{w_1}{w_1 + w_2} f_1 + \frac{w_2}{w_1 + w_2} f_2 \\ &= \bar{w}_1 f_1 + \bar{w}_2 f_2 \\ &= (\bar{w}_1 x) p_1 + (\bar{w}_1 y) q_1 + (\bar{w}_1) r_1 \\ &\quad + (\bar{w}_2 x) p_2 + (\bar{w}_2 y) q_2 + (\bar{w}_2) r_2 \end{aligned} \quad (24)$$

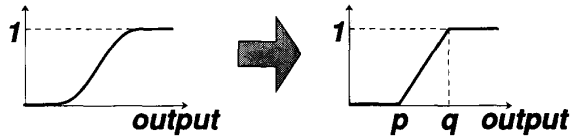


Fig. 7. Piecewise linear approximation of membership functions on the consequent part of type-1 ANFIS.

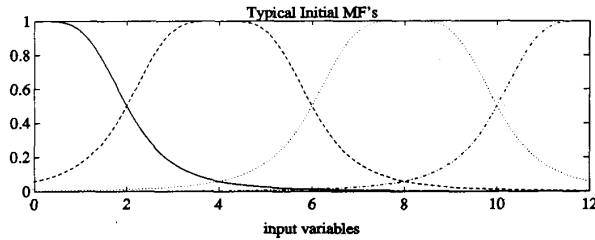


Fig. 8. A typical initial membership function setting in our simulation. (The operating range is assumed to be $[0, 12]$.)

which is linear in the consequent parameters (p_1 , q_1 , r_1 , p_2 , q_2 and r_2). As a result, we have

S = set of total parameters

S_1 = set of premise parameters

S_2 = set of consequent parameters

in (10); $H(\cdot)$ and $F(\cdot, \cdot)$ are the identity function and the function of the fuzzy inference system, respectively. Therefore the hybrid learning algorithm developed in the previous chapter can be applied directly. More specifically, in the forward pass of the hybrid learning algorithm, functional signals go forward till layer 4 and the consequent parameters are identified by the least squares estimate. In the backward pass, the error rates propagate backward and the premise parameters are updated by the gradient descent. Table I summarizes the activities in each pass.

As mentioned earlier, the consequent parameters thus identified are optimal (in the consequent parameter space) under the condition that the premise parameters are fixed. Accordingly the hybrid approach is much faster than the strict gradient descent and it is worthwhile to look for the possibility of decomposing the parameter set in the manner of (10). For type-1 ANFIS, this can be achieved if the membership function on the consequent part of each rule is replaced by a piecewise linear approximation with two consequent parameters (see Fig. 7). In this case, again, the consequent parameters constitute set S_2 and the hybrid learning rule can be employed directly.

However, it should be noted that the computation complexity of the least squares estimate is higher than that of the gradient descent. In fact, there are four methods to update the parameters, as listed below according to their computation complexities:

- 1) *Gradient Descent Only* : All parameters are updated by the gradient descent.
- 2) *Gradient Descent and One Pass of LSE*: The LSE is applied only once at the very beginning to get the

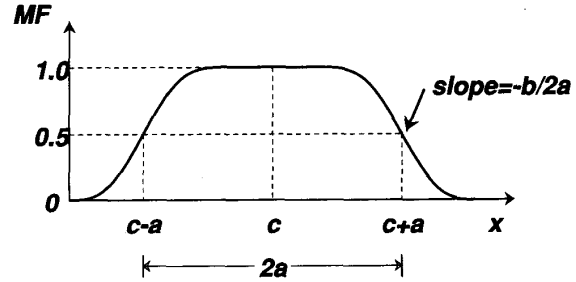


Fig. 9. Physical meanings of the parameters in the bell membership function $\mu_A(x) = 1/(1 + [(x - c/a)^2]^b)$.

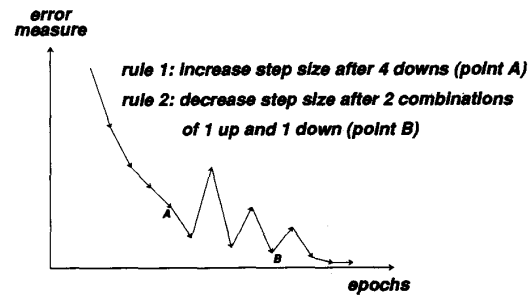


Fig. 10. Two heuristic rules for updating step size k .

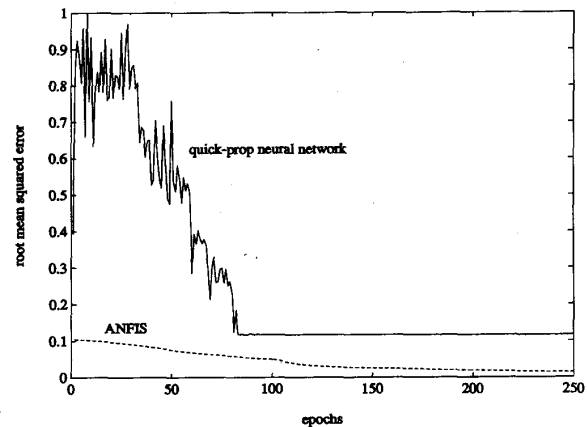


Fig. 11. RMSE curves for the quick-propagation neural networks and the ANFIS.

initial values of the consequent parameters and then the gradient descent takes over to update all parameters.

- 3) *Gradient descent and LSE* : This is the proposed hybrid learning rule.
- 4) *Sequential (Approximate) LSE Only*: The ANFIS is linearized w.r.t. the premise parameters and the extended Kalman filter algorithm is employed to update all parameters. This has been proposed in the neural network literature [41]–[43].

The choice of above methods should be based on the trade-off between computation complexity and resulting performance. Our simulations presented in the next section are performed

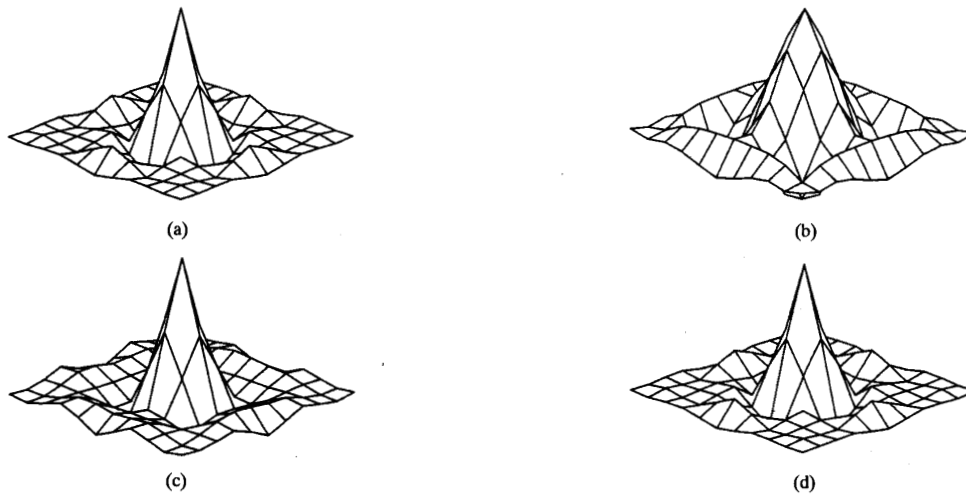


Fig. 12. Training data (a) and reconstructed surfaces at (b) 0.5, (c) 99.5, and 249.5 (d) epochs. (Example 1).

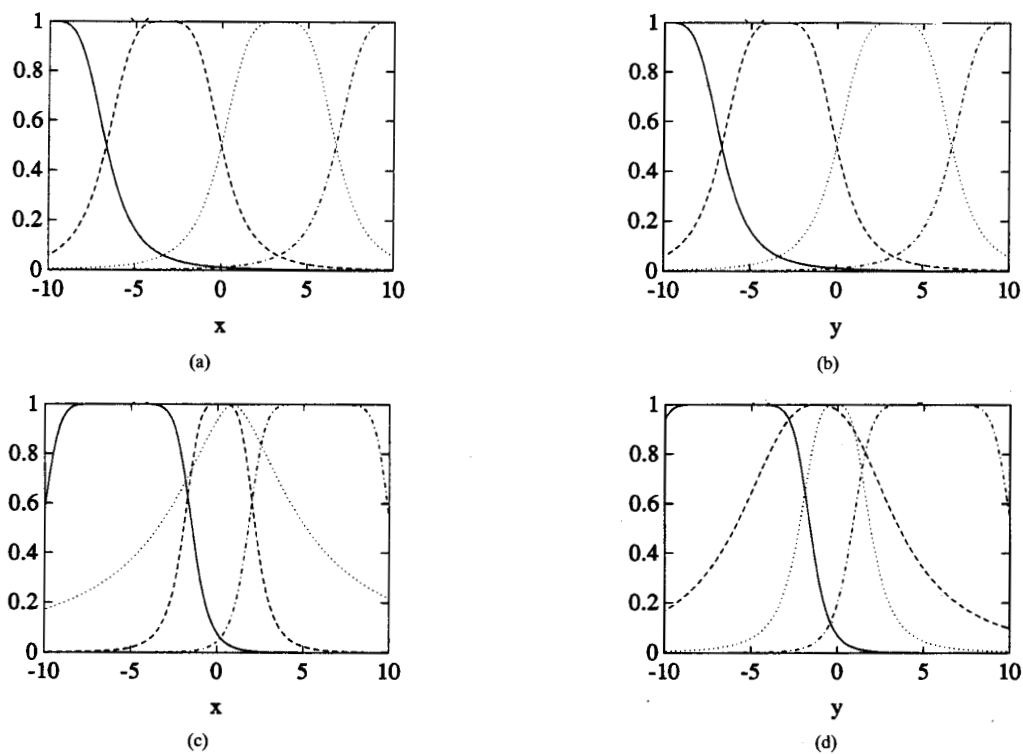


Fig. 13. Initial and final membership functions of example 1. (a) Initial MF's on x . (b) Initial MF's on y . (c) Final MF's on x . (d) Final MF's on y .

by the third method. Note that the consequent parameters can also be updated by the Widrow-Hoff LMS algorithm [63], as reported in [44]. The Widrow-Hoff algorithm requires less computation and favors parallel hardware implementation, but it converges relatively slowly when compared to the least square estimate.

As pointed out by one of the reviewers, the learning mechanisms should not be applied to the determination of membership functions since they convey linguistic and subjective description of ill-defined concepts. We think this is a case-by-case situation and the decision should be left to the users. In principle, if the size of available input-output data

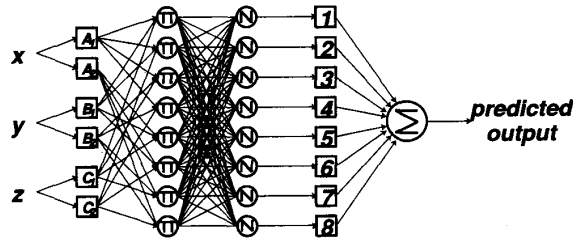


Fig. 14. The ANFIS architecture for example 2. (The connections from inputs to layer 4 are not shown.)

set is large enough, then the fine-tuning of the membership functions are applicable (or even necessary) since the human-determined membership functions are subject to the differences from person to person and from time to time; therefore they are rarely optimal in terms of reproducing desired outputs. However, if the data set is too small, then it probably does not contain enough information of the system under consideration. In this situation, the human-determined membership functions represent important knowledge obtained through human experts' experiences and it might not be reflected in the data set; therefore the membership functions should be kept fixed throughout the learning process.

Interestingly enough, if the membership functions are fixed and only the consequent part is adjusted, the ANFIS can be viewed as a functional-link network [19], [34] where the "enhanced representation" of the input variables are achieved by the membership functions. This "enhanced representation" which takes advantage of human knowledge are apparently more insight-revealing than the functional expansion and the tensor (outerproduct) models [34]. By fine-tuning the membership functions, we actually make this "enhanced representation" also adaptive.

Because the update formulas of the premise and consequent parameters are decoupled in the hybrid learning rule (see Table I), further speedup of learning is possible by using other versions of the gradient method on the premise parameters, such as conjugate gradient descent, second-order back-propagation [35], quick-propagation [5], nonlinear optimization [58] and many others.

C. Fuzzy Inference Systems with Simplified Fuzzy If-Then Rules

Though the reasoning mechanisms (see Fig. 2) introduced earlier are commonly used in the literature, each of them has inherent drawbacks. For type-1 reasoning (see Fig. 2 or 5), the membership functions on the consequence part are restricted to monotonic functions which are not compatible with linguistic terms such as "medium" whose membership function should be bell-shaped. For type-2 reasoning (see Fig. 2), the defuzzification process is time-consuming and systematic fine-tuning of the parameters are not easy. For type-3 reasoning (see Fig. 2 or 4), it is just hard to assign any appropriate linguistic terms to the consequence part which is a nonfuzzy function of the input variables. To cope with these disadvantages, simplified fuzzy if-then rules of the following form are introduced:

If x is big and y is small, then z is d.

where d is a crisp value. Due to the fact that the output z is described by a crisp value (or equivalently, a singular membership function), this class of simplified fuzzy if-then rules can employ all three types of reasoning mechanisms. More specifically, the consequent part of this simplified fuzzy if-then rule is represented by a step function (centered at $z = d$) in type 1, a singular membership function (at $z = d$) in type 2, and a constant output function in type 3, respectively. Thus the three reasoning mechanisms are unified under this simplified fuzzy if-then rules.

Most of all, with this simplified fuzzy if-then rule, it is possible to prove that under certain circumstance, the resulting fuzzy inference system has unlimited approximation power to match any nonlinear functions arbitrarily well on a compact set. We will proceed this in a descriptive way by applying the Stone-Weierstrass theorem [18], [38] stated below.

Theorem 1: Let domain D be a compact space of N dimensions, and let $\mathcal{F}$ be a set of continuous real-valued functions on D , satisfying the following criteria:

- 1) *Identity Function:* The constant $f(x) = 1$ is in $\mathcal{F}$.
- 2) *Separability:* For any two points $x_1 \neq x_2$ in D , there is an f in $\mathcal{F}$ such that $f(x_1) \neq f(x_2)$.
- 3) *Algebraic Closure:* If f and g are any two functions in $\mathcal{F}$, then fg and $af + bg$ are in $\mathcal{F}$ for any two real numbers a and b .

Then $\mathcal{F}$ is dense in $C(D)$, the set of continuous real-valued functions on D . In other words, for any $\epsilon > 0$, and any function g in $C(D)$, there is a function f in $\mathcal{F}$ such that $|g(x) - f(x)| < \epsilon$ for all $x \in D$.

In application of fuzzy inference systems, the domain in which we operate is almost always closed and bounded and therefore it is compact. For the first and second criteria, it is trivial to find simplified fuzzy inference systems that satisfy them. Now all we need to do is examine the algebraic closure under addition and multiplication. Suppose we have two fuzzy inference systems S and $\tilde{S}$; each has two rules and the output of each system can be expressed as

$$S : z = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2} \quad (25)$$

$$\tilde{S} : \tilde{z} = \frac{\tilde{w}_1 \tilde{f}_1 + \tilde{w}_2 \tilde{f}_2}{\tilde{w}_1 + \tilde{w}_2} \quad (26)$$

where $f_1, f_2, \tilde{f}_1$ and $\tilde{f}_2$ are constant output of each rule. Then $az + b\tilde{z}$ and $z\tilde{z}$ can be calculated as follows:

$$\begin{aligned} az + b\tilde{z} &= a \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2} + b \frac{\tilde{w}_1 \tilde{f}_1 + \tilde{w}_2 \tilde{f}_2}{\tilde{w}_1 + \tilde{w}_2} \\ &= \frac{w_1 \tilde{w}_1 (a f_1 + b \tilde{f}_1) + w_1 \tilde{w}_2 (a f_1 + b \tilde{f}_2)}{w_1 \tilde{w}_1 + w_1 \tilde{w}_2 + w_2 \tilde{w}_1 + w_2 \tilde{w}_2} \\ &\quad + \frac{w_2 \tilde{w}_1 (a f_2 + b \tilde{f}_1) + w_2 \tilde{w}_2 (a f_2 + b \tilde{f}_2)}{w_1 \tilde{w}_1 + w_1 \tilde{w}_2 + w_2 \tilde{w}_1 + w_2 \tilde{w}_2}, \\ z\tilde{z} &= \frac{w_1 \tilde{w}_1 f_1 \tilde{f}_1 + w_1 \tilde{w}_2 f_1 \tilde{f}_2 + w_2 \tilde{w}_1 f_2 \tilde{f}_1 + w_2 \tilde{w}_2 f_2 \tilde{f}_2}{w_1 \tilde{w}_1 + w_1 \tilde{w}_2 + w_2 \tilde{w}_1 + w_2 \tilde{w}_2} \end{aligned} \quad (27)$$

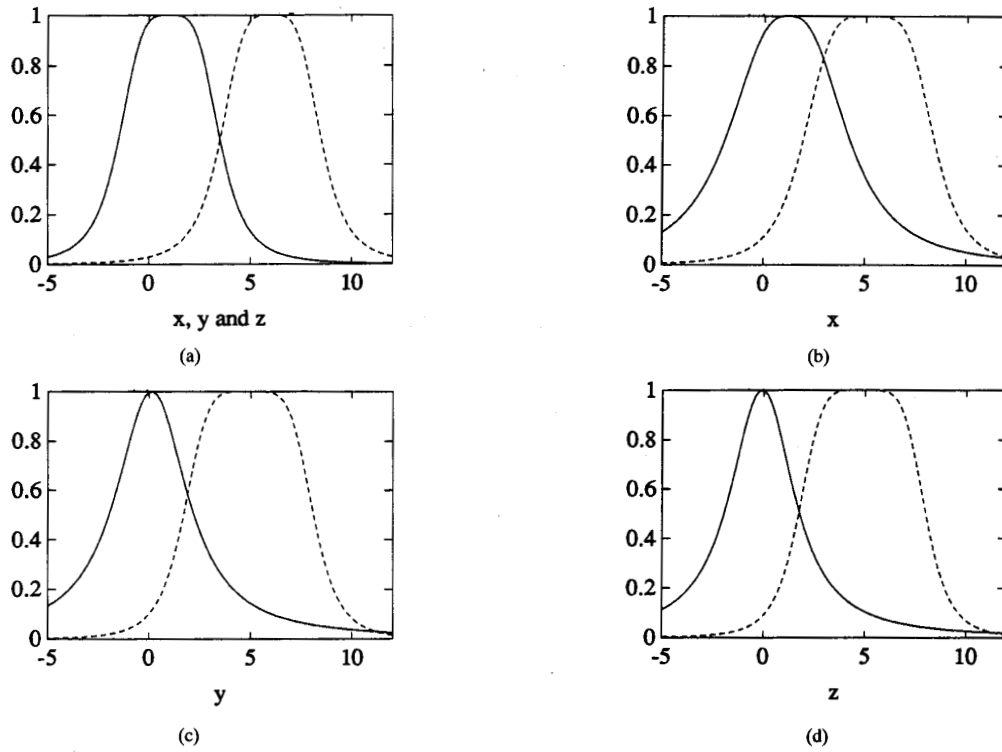


Fig. 15. Example 2. (a) Membership functions before learning. (b)–(d) Membership functions after learning. (a) Initial MF's on x , y , and z . (b) Final MF's on x . (c) Final MF's on y . (d) Final MF's on z .

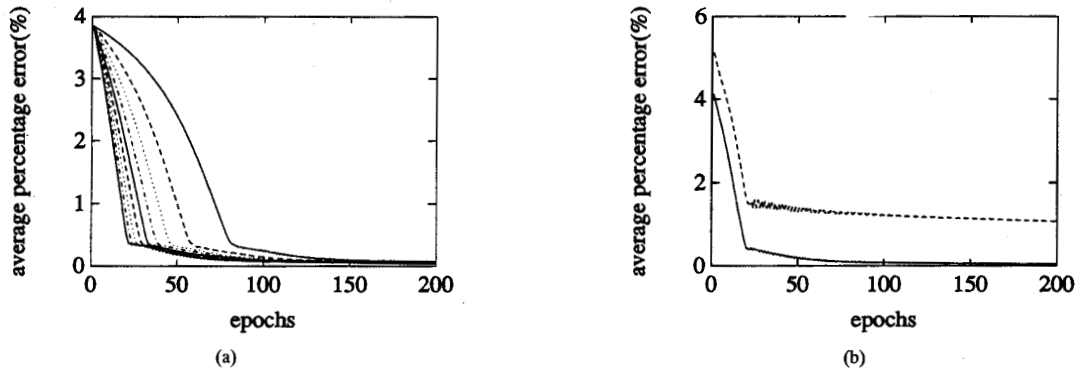


Fig. 16. Error curves of example 2: (a) Nine training error curves for nine initial step size from 0.01 (solid line) to 0.09. (b) training (solid line) and checking (dashed line) error curves with initial step size equal to 0.1.

which are of the same form as (25) and (26). Apparently the ANFIS architectures that compute $az + b\bar{z}$ and $z\bar{z}$ are of the same class of S and $\bar{S}$ if and only if the class of membership functions is invariant under multiplication. This is loosely true if the class of membership functions is the set of all bell-shaped functions, since the multiplication of two bell-shaped function is almost always still bell-shaped. Another more tightly defined class of membership functions satisfying this criteria, as pointed out by Wang [56], [57], is the scaled

Gaussian membership function:

$$\mu_{A_i}(x) = a_i \exp\left[-\left(\frac{x - c_i}{a_i}\right)^2\right] \quad (28)$$

Therefore by choosing an appropriate class of membership functions, we can conclude that the ANFIS with simplified fuzzy if-then rules satisfy the criteria of the Stone-Weierstrass theorem. Consequently, for any given $\epsilon > 0$, and any real-

TABLE II
EXAMPLE 2: COMPARISONS WITH EARLIER WORK

Model	APE_{trn} (%)	APE_{chk} (%)	Parameter Number	Training Set Size	Checking Set Size
ANFIS	0.043	1.066	50	216	125
GMDH model	4.7	5.7	—	20	20
Fuzzy model 1	1.5	2.1	22	20	20
Fuzzy model 2	0.59	3.4	32	20	20

TABLE III
EXAMPLE 3: COMPARISON WITH NN IDENTIFIER

Method	Parameter Number	Time Steps of Adaptation
NN	261	50 000
ANFIS	35	250

valued function g , there is a fuzzy inference system S such that $|g(\bar{x}) - S(\bar{x})| < \epsilon$ for all $\bar{x}$ in the underlying compact set. Moreover, since the simplified ANFIS is a proper subset of all three types of ANFIS in Fig. 2, we can draw the conclusion that all the three types of ANFIS have unlimited approximation power to match any given data set. However, caution has to be taken in accepting this claim since there is no mention about how to construct the ANFIS according to the given data set. That is why learning plays a role in this context.

Another interesting aspect of the simplified ANFIS architecture is its functional equivalence to the radial basis function network (RBFN). This functional equivalence is established when the Gaussian membership function is used in the simplified ANFIS. A detailed treatment can be found in [13]. This functional equivalence provides us with a shortcut for better understanding of ANFIS and RBFN and advances in either literatures apply to both directly. For instance, the hybrid learning rule of ANFIS can be apply to RBFN directly and, vice versa, the approaches used to identify RBFN parameters, such as clustering preprocess [29], [30], orthogonal least squares learning [3], generalization properties [2], sequential adaptation [15], among others [14], [31], are all applicable techniques for ANFIS.

V. APPLICATION EXAMPLES

This section presents the simulation results of the proposed type-3 ANFIS with both batch (off-line) and pattern (on-line) learning. In the first two examples, ANFIS is used to model highly nonlinear functions and the results are compared with neural network approach and earlier work. In the third example, ANFIS is used as an identifier to identify a nonlinear component on-line in a discrete control system. Lastly, we use ANFIS to predict a chaotic time series and compare the results with various statistical and connectionist approaches.

A. Practical Considerations

In a conventional fuzzy inference system, the number of rules is decided by an expert who is familiar with the system to be modeled. In our simulation, however, no expert is available and the number of membership functions (MF's) assigned to

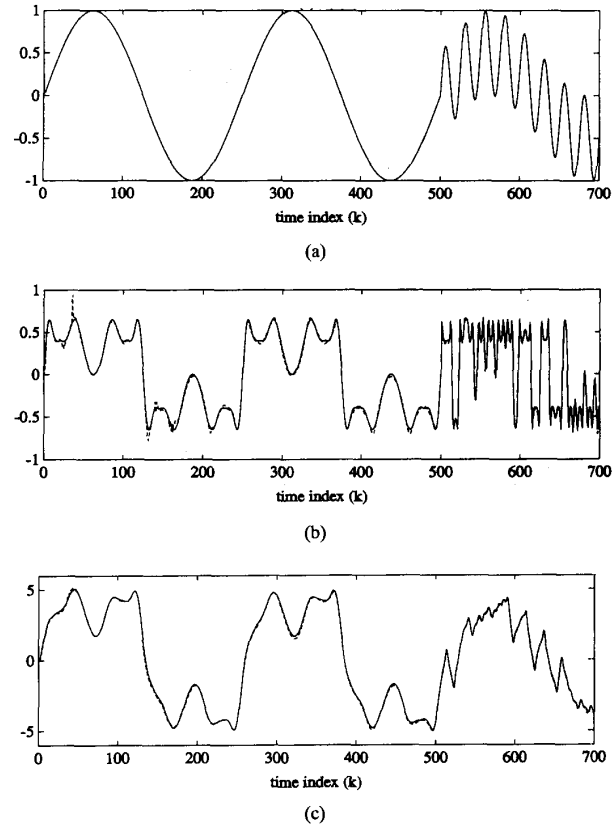


Fig. 17. Example 3. (a) $u(k)$. (b) $f(u(k))$ and $F(u(k))$. (c) Plant output and model output.

each input variable is chosen empirically, i.e., by examining the desired input-output data and/or by trial and error. This situation is much the same as that of neural networks; there are no simple ways to determine in advance the minimal number of hidden nodes necessary to achieve a desired performance level.

After the number of MF's associated with each inputs are fixed, the initial values of premise parameters are set in such a way that the MF's are equally spaced along the operating range of each input variable. Moreover, they satisfy ϵ -completeness [23], [24] with $\epsilon = 0.5$, which means that given a value x of one of the inputs in the operating range, we can always find a linguistic label A such that $\mu_A(x) \geq \epsilon$. In this manner, the fuzzy inference system can provide smooth transition and sufficient overlapping from one linguistic label to another. Though we did not attempt to keep the ϵ -completeness during

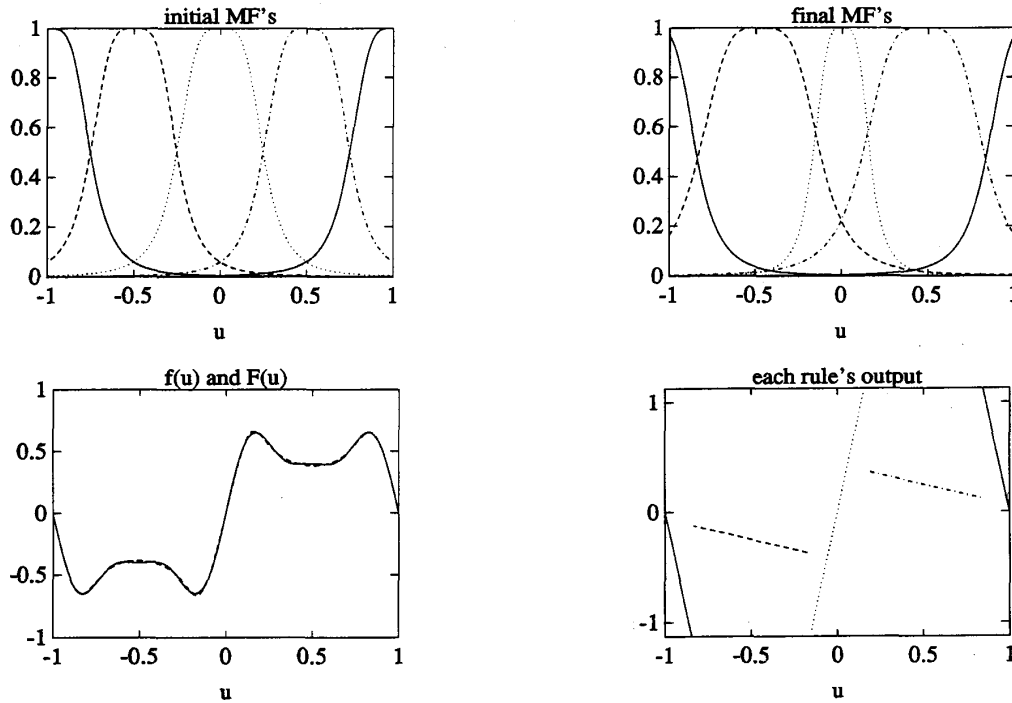


Fig. 18. Example 3: batch learning with five MF's.

the learning in our simulation, it can be easily achieved by using the constrained gradient method [65]. Fig. 8 shows a typical initial MF setting when the number of MF is 4 and the operating range is $[0, 12]$. Note that throughout the simulation examples presented below, all the membership functions used are the generalized bell function defined in (18):

$$\mu_A(x) = \frac{1}{1 + \left[\left(\frac{x-c}{a}\right)^2\right]^b} \quad (29)$$

which contains three fitting parameters a , b and c . Each of these parameters has a physical meaning: c determines the center of the corresponding membership function; a is the half width; and b (together with a) controls the slopes at the crossover points (where MF value is 0.5). Fig. 9 shows these concepts.

We mentioned that the step size k in (8) may influence the speed of convergence. It is observed that if k is small, the gradient method will closely approximate the gradient path, but convergence will be slow since the gradient must be calculated many times. On the other hand, if k is large, convergence will initially be very fast, but the algorithm will oscillate about the optimum. Based on these observations, we update k according to the following two heuristic rules (see Fig. 10):

- 1) If the error measure undergoes four consecutive reductions, increase k by 10%.
- 2) If the error measure undergoes two consecutive combinations of one increase and one reduction, decrease k by 10%.

Though the numbers 10%, 4 and 2 are chosen more or less arbitrarily, the results shown in our simulation appear to be satisfactory. Furthermore, due to this dynamical update strategy, the initial value of k is usually not critical as long as it is not too big.

B. Simulation Results

Example 1—Modeling a Two-Input Nonlinear Function: In this example, we consider using ANFIS to model a nonlinear sinc equation

$$z = \text{sinc}(x, y) = \frac{\sin(x)}{x} \times \frac{\sin(y)}{y}. \quad (30)$$

From the grid points of the range $[-10, 10] \times [-10, 10]$ within the input space of the above equation, 121 training data pairs were obtained first. The ANFIS used here contains 16 rules, with four membership functions being assigned to each input variable and the total number of fitting parameters is 72 which are composed of 24 premise parameters and 48 consequent parameters. (We also tried ANFIS with 4 rules and 9 rules, but obviously they are too simple to describe the highly nonlinear sinc function.)

Fig. 11 shows the RMSE (root mean squared error) curves for both the 2-18-1 neural network and the ANFIS. Each curve is the average of ten runs: for the neural network, this ten runs were started from 10 different set of initial random weights; for the ANFIS, 10 different initial step size ($= 0.01, 0.02, \dots, 0.10$) were used. The neural network, contain-

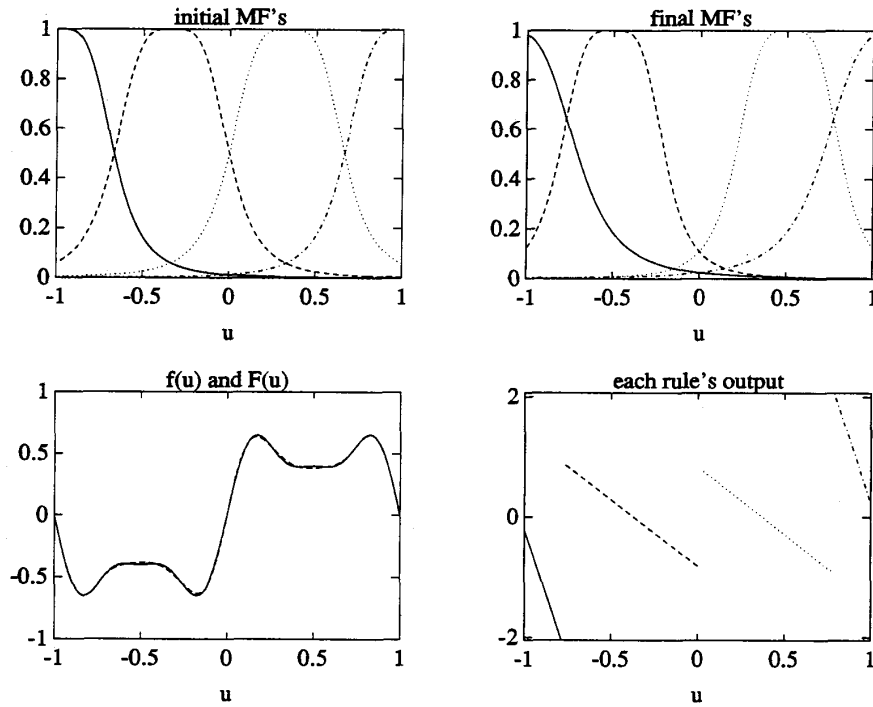


Fig. 19. Example 3: Batch learning with four MF's.

ing 73 fitting parameters (connection weights and thresholds), was trained with quick propagation [5] which is considered one of the best learning algorithms for connectionist models. Fig. 11 demonstrate how ANFIS can effectively model a highly nonlinear surface as compared to neural networks. However, this comparison cannot taken to be universal since we did not attempt an exhaustive search to find the optimal settings for the quick-propagation learning rule of the neural networks.

The training data and reconstructed surfaces at different epoch numbers are shown in Fig. 12. (Since the error measure is always computed after the forward pass is over, the epoch numbers shown in Fig. 12 always end with "0.5.") Note that the reconstructed surface after 0.5 epoch is due to the identification of consequent parameters only and it already looks similar to the training data surface.

Fig. 13 lists the initial and final membership functions. It is interesting to observe that the sharp changes of the training data surface around the origin is accounted for by the moving of the membership functions toward the origin. Theoretically, the final MF's on both x and y should be symmetric with respect to the origin. However, they are not symmetric due to the computer truncation errors and the approximate initial conditions for bootstrapping the calculation of the sequential least squares estimate in [14].

Example 2—Modeling a Three-Input Nonlinear Function: The training data in this example are obtained from

$$\text{output} = (1 + x^{0.5} + y^{-1} + z^{-1.5})^2, \quad (31)$$

which was also used by Takagi et al. [52], Sugeno et al. [47] and Kondo [20] to verify their approaches. The ANFIS (see Fig. 14) used here contains 8 rules, with 2 membership functions being assigned to each input variable. 216 training data and 125 checking data were sampled uniformly from the input ranges $[1, 6] \times [1, 6] \times [1, 6]$ and $[1.5, 5.5] \times [1.5, 5.5] \times [1.5, 5.5]$, respectively. The training data was used for the training of ANFIS, while the checking data was used for verifying the identified ANFIS only. To allow comparison, we use the same performance index adopted in [47, 20]:

$$\begin{aligned} APE &= \text{average percentage error} \\ &= \frac{1}{P} \sum_{i=1}^P \frac{|T(i) - O(i)|}{|T(i)|} * 100\%. \end{aligned} \quad (32)$$

where P is the number of data pairs; $T(i)$ and $O(i)$ are i th desired output and calculated output, respectively.

Fig. 15 illustrates the membership functions before and after training. The training error curves with different initial step sizes (from 0.01 to 0.09) are shown in Fig. 16(a), which demonstrates that the initial step size is not too critical on the final performance as long as it is not too big. Fig. 16(b) is the training and checking error curves with initial step size equal to 0.1. After 199.5 epochs, the final results are $APE_{trn} = 0.043\%$ and $APE_{chk} = 1.066\%$, which is listed in Table II along with other earlier work [47], [20]. Since each simulation cited here was performed under different assumptions and with different training and checking data sets, we cannot make conclusive comments here.

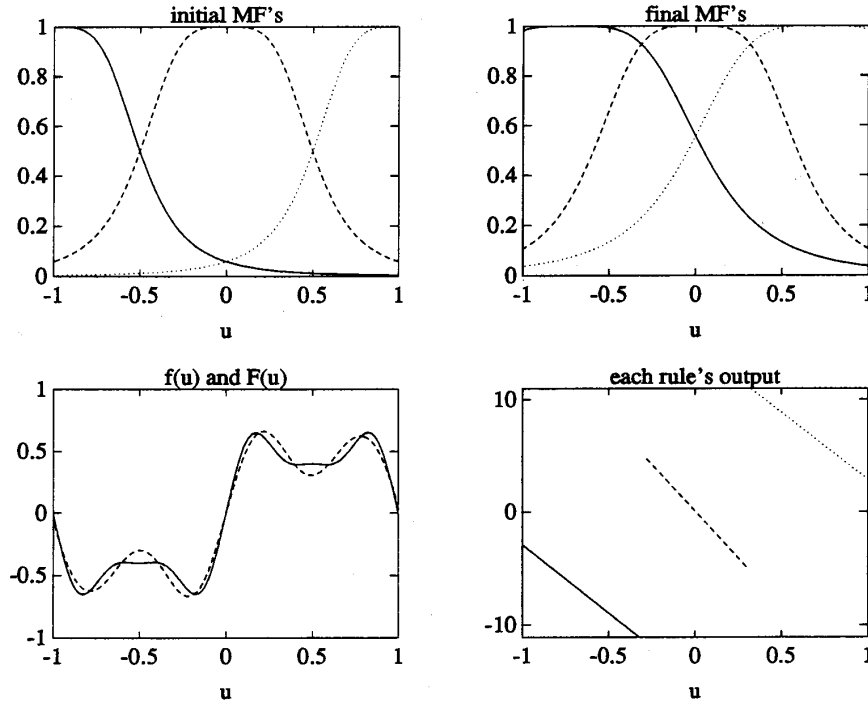


Fig. 20. Example 3: Batch learning with three MF's.

Example 3—On-line Identification in Control Systems: Here we repeat the simulation example 1 of [32] where a 1-20-10-1 neural network is employed to identify a nonlinear component in a control system, except that we use ANFIS to replace the neural network. The plant under consideration is governed by the following difference equation:

$$y(k+1) = 0.3y(k) + 0.6y(k-1) + f(u(k)), \quad (33)$$

where $y(k)$ and $u(k)$ are the output and input, respectively, at time index k , and the unknown function $f(\cdot)$ has the form

$$f(u) = 0.6 \sin(\pi u) + 0.3 \sin(3\pi u) + 0.1 \sin(5\pi u). \quad (34)$$

In order to identify the plant, a series-parallel model governed by the difference equation

$$\hat{y}(k+1) = 0.3\hat{y}(k) + 0.6\hat{y}(k-1) + F(u(k)) \quad (35)$$

was used where $F(\cdot)$ is the function implemented by ANFIS and its parameters are updated at each time index. Here the ANFIS has 7 membership functions on its input (thus 7 rules, and 35 fitting parameters) and the pattern (on-line) learning paradigm was adopted with a learning rate $\eta = 0.1$ and a forgetting factor $\lambda = 0.99$. The input to the plant and the model was a sinusoid $u(k) = \sin(2\pi k/250)$ and the adaptation started at $k = 1$ and stopped at $k = 250$. As shown in Fig. 17, the output of the model follows the output of the plant almost immediately even after the adaptation stopped at $k = 250$ and the $u(k)$ is changed to $0.5 \sin(2\pi k/250) + 0.5 \sin(2\pi k/25)$ after $k = 500$. As a comparison, the neural network in

[32] fails to follow the plant when the adaptation stopped at $k = 500$ and the identification procedure had to continue for 50,000 time steps using a random input. Table III summarizes the comparison.

In the above, the MF number is determined by trial and errors. If the MF number is below 7 then the model output will not follow the plant output satisfactorily after 250 adaptations. But can we decrease the parameter numbers by using batch learning which is supposed to be more effective? Fig. 18, 19 and 20 show the results after 49.5 epochs of batch learning when the MF numbers are 5, 4 and 3, respectively. As can be seen, the ANFIS is a good model even when the MF is as small as 3. However, as the MF number is getting smaller, the correlation between $F(u)$ and each rule's output is getting less obvious in the sense that it is harder to sketch $F(u)$ from each rule's consequent part. In other words, when the parameter number is reduced mildly, usually the ANFIS can still do the job but at the cost of sacrificing its semantics in terms of the local-description nature of fuzzy if-then rules; it is less of a structured knowledge representation and more of a black-box model (like neural networks).

Example 4—Predicting Chaotic Dynamics: Example 1–3 show that the ANFIS can be used to model highly nonlinear functions effectively. In this example, we will demonstrate how the proposed ANFIS can be employed to predict future values of a chaotic time series. The performance obtained in this example will be compared with the results of a cascade-correlation neural network approach reported in [37] and a

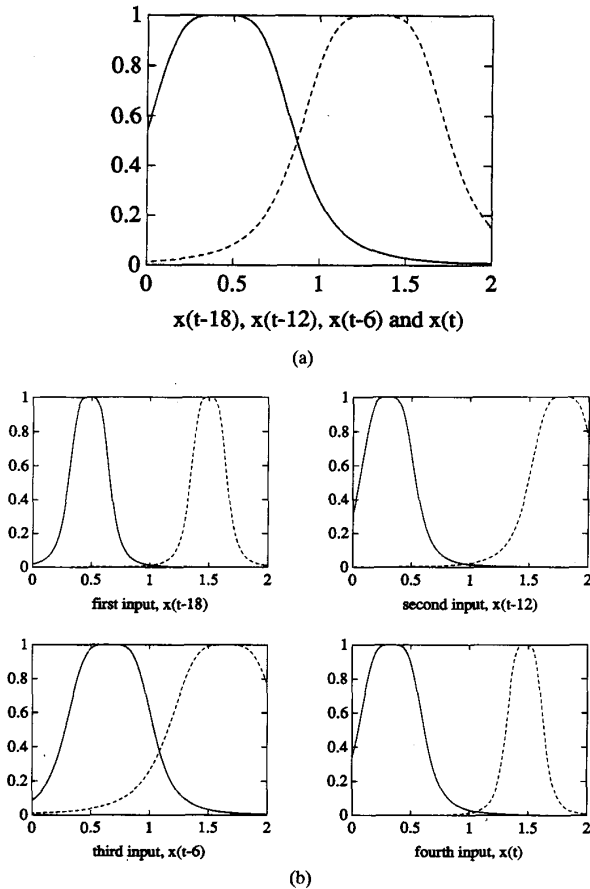


Fig. 21. Membership functions of example 4. (a) Before learning. (b) After learning.

simple conventional statistical approach, the auto-regressive (AR) model.

The time series used in our simulation is generated by the chaotic Mackey-Glass differential delay equation [27] defined below:

$$\dot{x}(t) = \frac{0.2x(t-\tau)}{1+x^{10}(t-\tau)} - 0.1x(t). \quad (36)$$

The prediction of future values of this time series is a benchmark problem which has been considered by a number of connectionist researchers (Lapedes and Farber [22], Moody [30], [28], Jones *et al.* [14], Crower [37] and Sanger [40]).

The goal of the task is to use known values of the time series up to the point $x = t$ to predict the value at some point in the future $x = t + P$. The standard method for this type of prediction is to create a mapping from D points of the time series spaced Δ apart, that is, $(x(t - (D-1)\Delta), \dots, x(t - \Delta), x(t))$, to a predicted future value $x(t + P)$. To allow comparison with earlier work (Lapedes and Farber [22], Moody [30, 28], Crower [37]), the values $D = 4$ and $\Delta = P = 6$ were used. All other simulation settings in this example were purposely arranged to be as close as possible to those reported in [37].

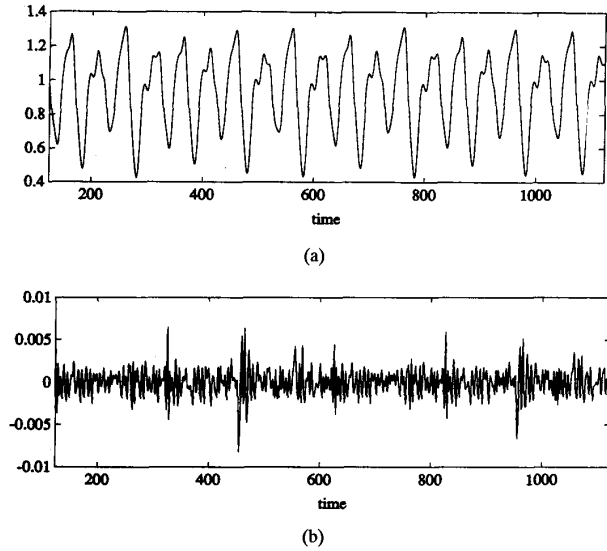


Fig. 22. Example 3. (a) Mackey-Glass time series from $t = 124$ to 1123 and six-step ahead prediction (which is indistinguishable from the time series here). (b) Prediction error.

To obtain the time series value at each integer point, we applied the fourth-order Runge-Kutta method to find the numerical solution to (36). The time step used in the method is 0.1, initial condition $x(0) = 1.2$, $\tau = 17$, and $x(t)$ is thus derived for $0 \leq t \leq 2000$. (We assume $x(t) = 0$ for $t < 0$ in the integration.) From the Mackey-Glass time series $x(t)$, we extracted 1000 input-output data pairs of the following format:

$$[x(t-18), x(t-12), x(t-6), x(t); x(t+6)], \quad (37)$$

where $t = 118$ to 1117. The first 500 pairs (training data set) was used for training the ANFIS while the remaining 500 pairs (checking data set) were used for validating the identified model. The number of membership functions assigned to each input of the ANFIS was arbitrarily set to 2, so the rule number is 16. Fig. 21(a) is the initial membership functions for each input variable. The ANFIS used here contains a total of 104 fitting parameters, of which 24 are premise parameters and 80 are consequent parameters.

After 499.5 epochs, we had $RMSE_{trn} = 0.0016$ and $RMSE_{chk} = 0.0015$, which are much better when compared with other approaches explained below. The resulting 16 fuzzy if-then rules are listed in the Appendix. The desired and predicted values for both training data and checking data are essentially the same in Fig. 22(a); their differences (see Fig. 22(b)) can only be seen on a finer scale. Fig. 21(b) is the final membership functions; Fig. 23 shows the RMSE curves which indicate most of the learning was done in the first 100 epochs. It is quite unusual to observe the phenomenon that $RMSE_{trn} < RMSE_{chk}$ during the training process. Considering both the RMSE's are very small, we conclude that: 1) the ANFIS has captured the essential components of the underlying dynamics; 2) the training data contains the effects of the initial conditions (remember that we set $x(t) = 0$ for $t \leq 0$ in the integration) which might not be easily

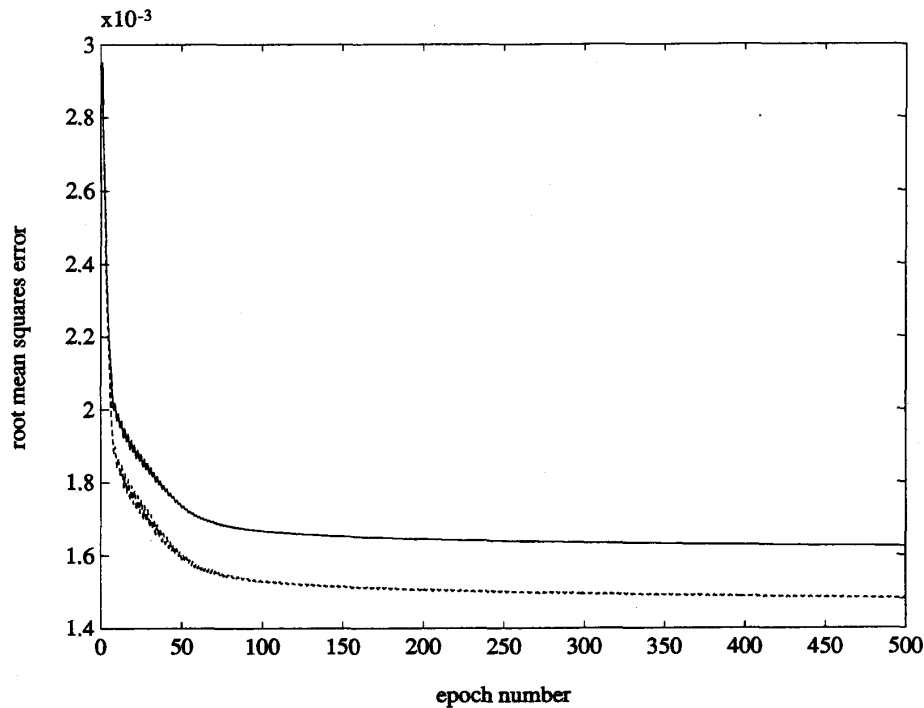


Fig. 23. Training and checking RMSE curves for ANFIS modeling.

accounted for by the essential components identified by the ANFIS.

As a comparison, we performed the same prediction by using the auto-regressive (AR) model with the same number of parameters:

$$x(t+6) = a_0 + a_1x(t) + a_2x(t-6) + \dots + a_{103}x(t-102 \cdot 6) \quad (38)$$

where there are 104 fitting parameters a_k , $k = 0$ to 103. From $t = 712$ to 1711, we extracted 1000 data pairs, of which the first 500 were used to identify a_k and the remaining were used for checking. The results obtained through the standard least squares estimate are $RMSE_{trn} = 0.005$ and $RMSE_{chk} = 0.078$ which is much worse than those of ANFIS. Fig. 24 shows the predicted values and the prediction errors. Obviously, the over-parameterization of the AR model causes over-fitting in the training data and large errors in the checking data. To search for the best AR model in terms of generalization capability, we tried out different AR models with parameter number being varied from 2 to 104; Fig. 25 shows the results where the AR model with the best generalization capability is obtained when the parameter number is 45. Based on this best AR model, we repeat the generalization test and Fig. 26 shows the results where there is no over-fitting at the price of larger training errors.

It goes without saying that the nonlinear ANFIS outperforms the linear AR model. However, it should be noted that the identification of the AR model took only a few seconds, while the ANFIS simulation took about 1.5 h on a HP Apollo 700

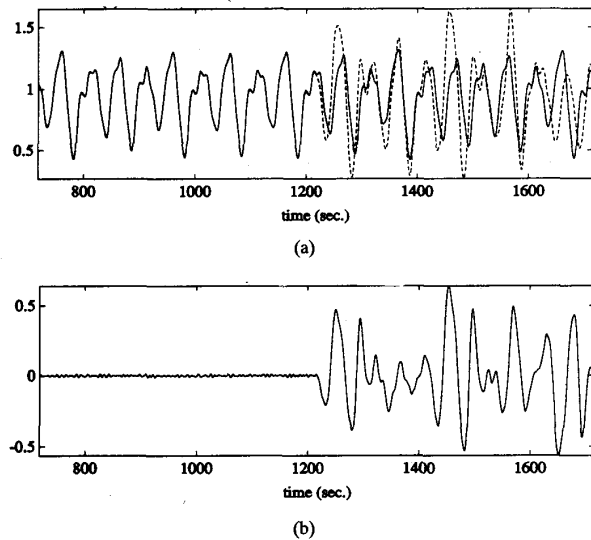


Fig. 24. (a) Mackey-Glass time series (solid line) from $t = 718$ to 1717 and six-step ahead prediction (dashed line) by AR model with parameter = 104. (b) Prediction errors.

Series workstation. (We did not pay special attention on the optimization of the codes, though.)

Table IV lists other methods' generalization capabilities which are measured by using each method to predict 500 points immediately following the training set. Here the non-

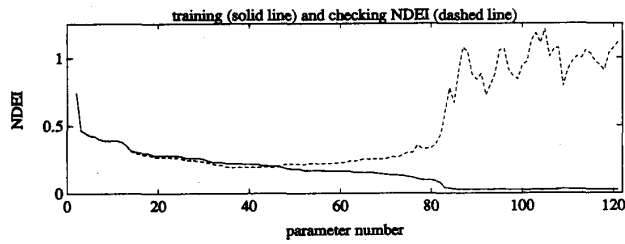


Fig. 25. Training (solid line) and checking (dashed line) errors of AR models with different parameter numbers.

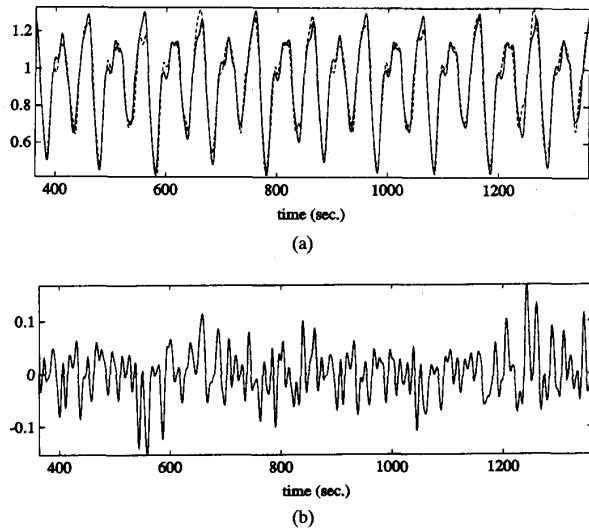


Fig. 26. Example 3. (a) Mackey-Glass time series (solid line) from $t = 364$ to 1363 and six-step ahead prediction (dashed line) by the best AR model (parameter number = 45). (b) Prediction errors.

TABLE IV
GENERALIZATION RESULT COMPARISONS FOR $P = 6^a$

Method	Training Cases	Non-Dimensional Error Index
ANFIS	500	0.007
AR Model	500	0.19
Cascaded-Correlation NN	500	0.06
Back-Prop NN	500	0.02
Sixth-order Polynomial	500	0.04
Linear Predictive Method	2000	0.55

dimensional error index (NDEI) [22], [37] is defined as the root mean square error divided by the standard deviation of the target series. (Note that the *average relative variance* used in [59, 60] is equal to the square of NDEI.) The remarkable generalization capability of the ANFIS, we believe, comes from the following facts:

- 1) The ANFIS can achieve a highly nonlinear mapping as shown in Example 1, 2 and 3, therefore it is superior to common linear methods in reproducing nonlinear time series.

TABLE V
GENERALIZATION RESULT COMPARISONS^a

Method	Training Cases	Non-Dimensional Error Index
ANFIS	500	0.036
AR Model	500	0.39
Cascaded-Correlation NN	500	0.32
Back-Prop NN	500	0.05
Sixth-Order Polynomial	500	0.85
Linear Predictive Method	2000	0.60
LRF	500	0.10–0.25
LRF	10000	0.025–0.05
MRH	500	0.05
MRH	10000	0.02

^a Generalization result comparisons for $P = 84$ (the first six rows) and 85 (the last four rows). Results for the first six methods are generated by iterating the solution at $P = 6$. Results for localized receptive fields (LRF) are multiresolution hierarchies (MRH) are for networks trained for $P = 85$. (The last eight rows are from [37].)

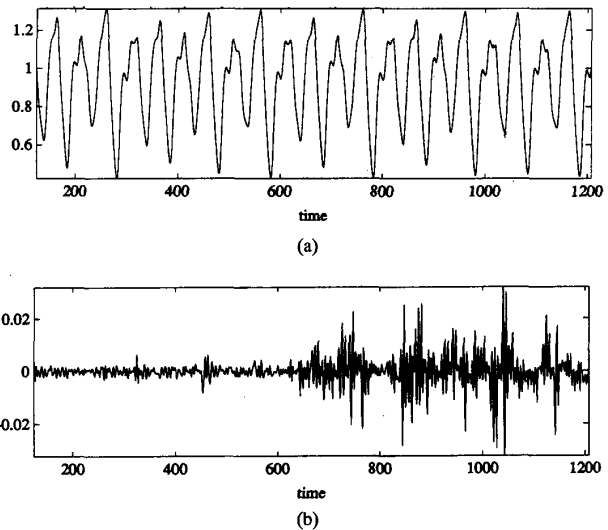


Fig. 27. Generalization test of ANFIS for $P = 84$. (a) Desired (solid) and predicted (dashed) time series of ANFIS when $P = 84$. (b) Prediction errors.

- 2) The ANFIS used here has 104 adjustable parameters, much less than those of the cascade-correlation NN (693, the median size) and back-prop NN (about 540) listed in Table IV.
- 3) Though without *a priori* knowledge, the initial parameter settings of ANFIS are intuitively reasonable and it leads to fast learning that captures the underlying dynamics.

Table V lists the results of the more challenging generalization test when $P = 84$ (the first six rows) and $P = 85$ (the last four rows). The results of the first six rows were obtained by iterating the prediction of $P = 6$ till $P = 84$. ANFIS still outperforms these statistical and connectionist approaches unless a substantially large amount of training data (i.e., the last row of Table V) were used instead. Fig. 27 illustrates the generalization test for the ANFIS where the first 500 points

$$\left\{ \begin{array}{l}
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_1 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_2 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_3 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_4 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_5 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_6 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_7 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{SMALL}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_8 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_9 \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_{10} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_{11} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{SMALL}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_{12} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_{13} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{SMALL}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_{14} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{SMALL}_4, \text{ then } x(t+6) = \tilde{c}_{15} \cdot \tilde{X} \\
\text{If } x(t-18) \text{ is } \text{LARGE}_1 \text{ and } x(t-12) \text{ is } \text{LARGE}_2 \text{ and } x(t-6) \text{ is } \text{LARGE}_3 \text{ and } x(t) \text{ is } \text{LARGE}_4, \text{ then } x(t+6) = \tilde{c}_{16} \cdot \tilde{X}
\end{array} \right.$$

Fig. 28.

were used for the desired outputs while the last 500 are the predicted outputs for $P = 84$.

VI. CONCLUSION

A. Summary and Extensions of Current Work

We have described the architecture of adaptive-network-based fuzzy inference systems (ANFIS) with type-1 and type-3 reasoning mechanisms. By employing a hybrid learning procedure, the proposed architecture can refine fuzzy if-then rules obtained from human experts to describe the input-output behavior of a complex system. However, if human expertise is not available, we can still set up intuitively reasonable initial membership functions and start the learning process to generate a set of fuzzy if-then rules to approximate a desired data set, as shown in the simulation examples of nonlinear function modeling and chaotic time series prediction.

Due to the high flexibility of adaptive networks, the ANFIS can have a number of variants from what we have proposed here. For instance, the membership functions can be changed to *L-R representation* [4] which could be asymmetric. Furthermore, we can replace Π nodes in layer 2 with the parameterized *T-norm* [4] and let the learning rule to decide the best T-norm operator for a specific application. By employing the adaptive network as a common framework, we have also proposed other adaptive fuzzy models tailored for data classification [49], [50] and feature extraction [51] purposes.

Another important issue in the training of ANFIS is how to preserve the human-plausible features such as bell-shaped membership functions, ϵ -completeness [23], [24] or sufficient overlapping between adjacent membership functions, minimal uncertainty, etc. Though we did not pursue along this direction in this paper, mostly it can be achieved by maintaining certain constraints and/or modifying the original error measure as explained below.

- To keep bell-shaped membership functions, we need the membership functions to be bell-shaped regardless of the

parameter values. In particular, (18) and (19) become upside-down bell-shaped if $b_i < 0$; one easy way to correct this is to replace b_i with b_i^2 in both equations.

- The ϵ -completeness can be maintained by the constrained gradient descent [65]. For instance, suppose that $\epsilon = 0.5$ and the adjacent membership functions are of the form of (18) with parameter sets $\{a_i, b_i, c_i\}$ and $\{a_{i+1}, b_{i+1}, c_{i+1}\}$. Then the ϵ -completeness is satisfied if $c_i + a_i = c_{i+1} - a_{i+1}$ and this can be ensured throughout the training if the constrained gradient descent is employed.
- Minimal uncertainty refers to the situation that within most region of the input space, there should be a dominant fuzzy if-then rule to account for the final output, instead of multiple rules with similar firing strengths. This minimizes the uncertainty and make the rule set more informative. One way to do this is to use a modified error measure

$$E' = E + \beta \sum_{i=1}^P [-\bar{w}_i \times \ln(\bar{w}_i)] \quad (39)$$

where E is the original squared error; β is a weighting constant; P is the size of training data set; $\bar{w}_i$ is the normalized firing strength of the i th rule (see (21)) and $\sum_{i=1}^P [-\bar{w}_i \times \ln(\bar{w}_i)]$ is the *information entropy*. Since this modified error measure is not based on data fitting along, the ANFIS thus trained can also have a potentially better generalization capability. (However, due to this new error measure, the training should be based on the gradient descent alone.) The improvement of generalization by using an error measure based on both data fitting and weight elimination has been reported in the neural network literature [59], [60].

In this paper, we assume the structure of the ANFIS is fixed and the *parameter identification* is solved through the hybrid

TABLE VI
Table of premise parameters in example 4.

A	a	b	c
SMALL ₁	0.1790	2.0456	0.4798
LARGE ₁	0.1584	2.0103	1.4975
SMALL ₂	0.2410	1.9533	0.2960
LARGE ₂	0.2923	1.9178	1.7824
SMALL ₃	0.3798	2.1490	0.6599
LARGE ₃	0.4884	1.8967	1.6465
SMALL ₄	0.2815	2.0170	0.3341
LARGE ₄	0.1616	2.0165	1.4727

learning rule. However, to make the whole approach more complete, the *structure identification* [47], [48] (which concerns with the selection of an appropriate input-space partition style and the number of membership functions on each input, etc.) is equally important to the successful applications of ANFIS. Effective partition of the input space can decrease the rule number and thus increase the speed in both learning and application phases. Advances on neural networks' structure identification [6], [25] can shed some lights on this aspect.

B. Applications to Automatic Control and Signal Processing

Fuzzy control is by far the most successful applications of the fuzzy set theory and fuzzy inference systems. Due to the adaptive capability of ANFIS, its applications to adaptive control and learning control are immediate. Most of all, it can replace almost any neural networks in control systems to serve the same purposes. For instance, Narendra's pioneering work of using neural networks in adaptive control [32] can be all achieved similarly by ANFIS. Moreover, four of the generic designs (i.e., *supervised control*, *direct inverse control*, *neural adaptive control* and *back-propagation of utility*) of neural networks in control, as proposed by Werbos [9], [62], are also directly applicable schemes for ANFIS. Particularly we have employed a similar method of the *back-propagation through time* [33] or *unfolding in time* to achieve a self-learning fuzzy controller with four rules that can balance an inverted pendulum in a near-optimal manner [12]. It is expected that the advances of neural network techniques in control can promote those of ANFIS as well, and vice versa.

The active role of neural networks in signal processing [64], [21] also suggests similar applications of ANFIS. The non-linearity and structured knowledge representation of ANFIS are the primary advantages over classical linear approaches in adaptive filtering [8] and adaptive signal processing [63], such as identification, inverse modeling, predictive coding, adaptive channel equalization, adaptive interference (noise or echo) canceling, etc.

ACKNOWLEDGMENT

The author wish to thank the anonymous reviewers for their valuable comments. The guidance and help of Professor Lotfi A. Zadeh and other members of the "fuzzy group" at University of California at Berkeley are also gratefully acknowledged.

APPENDIX

As suggested by one of the reviewers, to give the readers a concrete idea of the resulting fuzzy inference systems, it would be better to list the fuzzy if-then rules explicitly. Here we list the final 16 fuzzy if-then rules in example 4, which predicts the Mackey-Glass chaotic time series. Suppose that the i th input variable is assigned two linguistic values $SMALL_i$ and $LARGE_i$, then the fuzzy if-then rules after training can be expressed as shown in Fig. 28, where $\vec{X} = [x(t-18), x(t-12), x(t-6), x(t), 1]$ and $\vec{c}_i$ is the i th row of the following consequent parameter matrix C :

$$C = \begin{bmatrix} 0.2167 & 0.7233 & -0.0365 & 0.5433 & 0.0276 \\ 0.2141 & 0.5704 & -0.4826 & 1.2452 & -0.3778 \\ -0.0683 & 0.0022 & 0.6495 & 2.7320 & -2.2916 \\ -0.2616 & 0.9190 & -2.9931 & 1.9467 & 1.6555 \\ -0.3293 & -0.8943 & 1.4290 & -1.6550 & 2.3735 \\ 2.5820 & -2.3109 & 3.7925 & -5.8068 & 4.0478 \\ 0.8797 & -0.9407 & 2.2487 & 0.7759 & -2.0714 \\ -0.8417 & -1.5394 & -1.5329 & 2.2834 & 2.4140 \\ -0.6422 & -0.4384 & 0.9792 & -0.3993 & 1.5593 \\ 1.5534 & -0.0542 & -4.7256 & 0.7244 & 2.7350 \\ -0.6864 & -2.2435 & 0.1585 & 0.5304 & 3.5411 \\ -0.3190 & -1.3160 & 0.9689 & 1.4887 & 0.7079 \\ -0.3200 & -0.4654 & 0.4880 & -0.0559 & 0.9622 \\ 4.0220 & -3.8886 & 1.0547 & -0.7427 & -0.4464 \\ 0.3338 & -0.3306 & -0.5961 & 1.1220 & 0.3529 \\ -0.5572 & 0.9190 & -0.8745 & 2.1899 & -0.9497 \end{bmatrix} \quad (41)$$

The linguistic labels $SMALL_i$ and $LARGE_i$ ($i=1$ to 4) are defined by the bell membership function (with different parameters a , b and c):

$$\mu_A(x) = \frac{1}{1 + \left[\left(\frac{x-c}{a}\right)^2\right]^b}. \quad (42)$$

These membership functions are shown in Fig. 21. Table VI lists the linguistic labels and the corresponding consequent parameters in (42).

REFERENCES

- [1] K. J. Aström and B. Wittenmark, *Computer Controller Systems: Theory and Design*. Prentice-Hall, 1984.
- [2] S. M. Botros and C. G. Atkeson, "Generalization properties of radial basis functions," in *Advances in Neural Information Processing Systems III*, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1991, pp. 707-713.
- [3] S. Chen, C. F. N. Cowan, and P. M. Grant, "Orthogonal least squares learning algorithm for radial basis function networks," *IEEE Trans. Neural Networks*, vol. 2, no. 2, pp. 302-309, Mar. 1991.
- [4] D. Dubois and H. Prade, *Fuzzy Sets and Systems: Theory and Applications*. New York: Academic, 1980.
- [5] S. E. Fahlman, "Faster-learning variations on back-propagation: an empirical study," in *Proc. 1988 Connectionist Models Summer School*, D. Touretzky, G. Hinton, and T. Sejnowski, Eds., Carnegie Mellon Univ., 1988, pp. 38-51.
- [6] S. E. Fahlman and C. Lebiere, "The cascade-correlation learning architecture," in *Advances in Neural Information Processing Systems II*, D. S. Touretzky, G. Hinton, and T. Sejnowski, Eds. San Mateo, CA: Morgan Kaufmann, 1990.
- [7] G. C. Goodwin and K. S. Sin, *Adaptive Filtering Prediction and Control*. Englewood Cliffs, NJ: Prentice-Hall, 1984.
- [8] S. S. Haykin, *Adaptive Filter Theory*. Englewood Cliffs, NJ: Prentice Hall, second ed., 1991.

- [9] W. T. Miller III, R. S. Sutton, and P. J. Werbos, Eds., *Neural Networks for Control*. Cambridge, MA: MIT Press, 1990.
- [10] J.-S. Roger Jang, "Fuzzy modeling using generalized neural networks and Kalman filter algorithm," in *Proc. Ninth Nat. Conf. Artificial Intell. (AAAI-91)*, July 1991, pp. 762-767.
- [11] —, "Rule extraction using generalized neural networks," in *Proc. 4th IFSA World Congress*, July 1991.
- [12] —, "Self-learning fuzzy controller based on temporal back-propagation," *IEEE Trans. Neural Networks*, Sept. 1992.
- [13] J.-S. Roger Jang and C.-T. Sun, "Functional equivalence between radial basis function networks and fuzzy inference systems," *IEEE Trans. Neural Networks*, vol. 4, pp. 156-159, Jan. 1993.
- [14] R. D. Jones, Y. C. Lee, C. W. Barnes, G. W. Flake, K. Lee, and P. S. Lewis, "Function approximation and time series prediction with neural networks," in *Proc. IEEE Int. Joint Conf. Neural Networks*, 1990, pp. 1-649-665.
- [15] V. Kadirkamanathan, M. Niranjan, and F. Fallside, "Sequential adaptation of radial basis function neural networks," in *Advances in Neural Information Processing Systems III*, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1991, pp. 721-727.
- [16] A. Kandel, *Fuzzy Expert Systems*. Reading, MA: Addison-Wesley, 1988.
- [17] A. Kandel, *Fuzzy Expert Systems*. Boca Raton, FL: CRC Press, 1992.
- [18] L. V. Kantorovich and G. P. Akilov, *Functional Analysis*, second edition. Oxford, UK: Pergamon, 1982.
- [19] M. S. Klassen and Y.-H. Pao, "Characteristics of the functional-link net: A higher order delta rule net," in *IEEE Proc. Int. Conf. Neural Networks*, San Diego, June 1988.
- [20] T. Kondo, "Revised GMDH algorithm estimating degree of the complete polynomial," *Trans. Soc. Instrument and Contr. Engineers*, vol. 22, no. 9, pp. 928-934, 1986 (in Japanese).
- [21] B. Kosko, *Neural networks for signal processing*. Englewood Cliffs, NJ: Prentice Hall, 1991.
- [22] A. S. Lapedes and R. Farber, "Nonlinear signal processing using neural networks: prediction and system modeling," Tech. Rep. LA-UR-87-2662, Los Alamos Nat. Lab., Los Alamos, NM, 1987.
- [23] C.-C. Lee, "Fuzzy logic in control systems: Fuzzy logic controller—Part I," *IEEE Trans. Syst., Man, Cybern.*, vol. 20, pp. 404-418, 1990.
- [24] —, "Fuzzy logic in control systems: Fuzzy logic controller—Part II," *IEEE Trans. Syst., Man, Cybern.*, vol. 20, pp. 419-435, 1990.
- [25] T.-C. Lee, *Structure Level Adaptation for Artificial Neural Networks*. Boston: Kluwer Academic, 1991.
- [26] L. Ljung, *System Identification: Theory for the User*. Englewood Cliffs, NJ: Prentice-Hall, 1987.
- [27] M. C. Mackey and L. Glass, "Oscillation and chaos in physiological control systems," *Science*, vol. 197, pp. 287-289, July 1977.
- [28] J. Moody, "Fast learning in multi-resolution hierarchies," in *Advances in Neural Information Processing Systems I*, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1989, ch. 1, pp. 29-39.
- [29] J. Moody and C. Darken, "Learning with localized receptive fields," in D. Touretzky, G. Hinton, and T. Sejnowski, Eds., in *Proc. 1988 Connectionist Models Summer School*, Carnegie Mellon University, Pittsburgh, PA. San Mateo, CA: Morgan Kaufmann Publishers, 1988.
- [30] —, "Fast learning in networks of locally-tuned processing units," *Neural Computation*, vol. 1, pp. 281-294, 1989.
- [31] M. T. Musavi, W. Ahmed, K. H. Chan, K. B. Faris, and D. M. Hummels, "On the training of radial basis function classifiers," *Neural Networks*, vol. 5, no. 4, pp. 595-603, 1992.
- [32] K. S. Narendra and K. Parthasarathy, "Identification and control of dynamical systems using neural networks," *IEEE Trans. Neural Networks*, vol. 1, no. 1, pp. 4-27, 1990.
- [33] D. H. Nguyen and B. Widrow, "Neural networks for self-learning control systems," *IEEE Contr. Syst. Mag.*, Apr. 1990, pp. 18-23.
- [34] Y.-H. Pao, *Adaptive Pattern Recognition and Neural Networks*. Reading, MA: Addison-Wesley, 1989, ch. 8, pp. 197-222.
- [35] D. B. Parker, "Optimal algorithms for adaptive networks: Second order back propagation, second order direct propagation, and second order Hebbian learning," in *Proc. IEEE Int. Conf. Neural Networks*, 1987, pp. 593-600.
- [36] W. Pedrycz, *Fuzzy Control and Fuzzy Systems*. New York: Wiley, 1989.
- [37] R. S. Crowder, "Predicting the Mackey-Glass timeseries with cascade-correlation learning," in *Proc. 1990 Connectionist Models Summer School*, D. Touretzky, G. Hinton, and T. Sejnowski, Eds., Carnegie Mellon Univ., 1990, pp. 117-123.
- [38] H. L. Royden, *Real Analysis*, second ed.. New York: Macmillan, 1968.
- [39] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, D. E. Rumelhart and James L. McClelland, Eds., vol. 1, ch. 8, pp. 318-362. Cambridge, MA: MIT Press, 1986.
- [40] T. D. Sanger, "A tree-structured adaptive network for function approximation in high-dimensional spaces," *IEEE Trans. Neural Networks*, vol. 2, no. 2, pp. 285-293, Mar. 1991.
- [41] S. Shah, F. Palmieri, and M. Datum, "Optimal filtering algorithms for fast learning in feedforward neural networks," *Neural Networks*, vol. 5, no. 5, pp. 779-787, 1992.
- [42] S. Shar and F. Palmieri, "MEKA—a fast, local algorithm for training feedforward neural networks," in *Proc. Int. Joint Conf. Neural Networks*, pp. III 41-46, 1990.
- [43] S. Singhal and L. Wu, "Training multilayer perceptrons with the extended kalman algorithm," *Advances in Neural Information Processing Systems I*, in D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1989, pp. 133-140.
- [44] S. M. Smith and D. J. Comer, "Automated calibration of a fuzzy logic controller using a cell state space algorithm," *IEEE Contr. Syst. Mag.*, vol. 11, no. 5, pp. 18-28, Aug. 1991.
- [45] P. Strobach, *Linear Prediction Theory: A Mathematical Basis for Adaptive Systems*. New York: Springer-Verlag, 1990.
- [46] M. Sugeno, Ed., *Industrial Applications of Fuzzy Control*. New York: Elsevier, 1985.
- [47] M. Sugeno and G. T. Kang, "Structure identification of fuzzy model," *Fuzzy Sets Syst.*, vol. 28, pp. 15-33, 1988.
- [48] C.-T. Sun, "Rulebase structure identification in an adaptive network based fuzzy inference system," *IEEE Trans. Fuzzy Syst.*, accepted for publication, 1993.
- [49] C.-T. Sun and J.-S. Roger Jang, "Adaptive network based fuzzy classification," in *Proc. Japan-U.S.A. Symp. Flexible Automat.*, July 1992.
- [50] —, "Fuzzy classification based on adaptive networks and genetic algorithms," submitted for publication in *IEEE Trans. Neural Networks*, 1992.
- [51] C.-T. Sun, J.-S. Roger Jang, and C.-Y. Fu, "Neural network analysis of plasma spectra," in *Proc. Int. Conf. Artificial Neural Networks*, Amsterdam, The Netherlands, 1993.
- [52] H. Takagi and I. Hayashi, "NN-driven fuzzy reasoning," *Int. J. Approximate Reasoning*, vol. 5, no. 3, pp. 191-212, 1991.
- [53] T. Takagi and M. Sugeno, "Derivation of fuzzy control rules from human operator's control actions," in *Proc. IFAC Symp. Fuzzy Inform., Knowledge Representation and Decision Analysis*, July 1983, pp. 55-60.
- [54] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Syst., Man, Cybern.*, vol. 15, pp. 116-132, 1985.
- [55] Y. Tsukamoto, "An approach to fuzzy reasoning method," in M. M. Gupta, R. K. Ragade, and R. R. Yager, Eds., *Advances in Fuzzy Set Theory and Applications*. Amsterdam: North-Holland, 1979, pp. 137-149.
- [56] L.-X. Wang, "Fuzzy systems are universal approximators," in *Proc. IEEE Int. Conf. Fuzzy Systems*, San Diego, CA, Mar. 1992.
- [57] L.-X. Wang and J. M. Mendel, "Fuzzy basis function, universal approximation, and orthogonal least squares learning," *IEEE Trans. Neural Networks*, vol. 3, no. 5, pp. 807-814, Sept. 1992.
- [58] R. L. Watrous, "Learning algorithms for connectionist network: applied gradient methods of nonlinear optimization," in *Proc. IEEE Int. Conf. Neural Networks*, 1991, pp. 619-627.
- [59] A. A. Weigend, D. E. Rumelhart, and B. A. Huberman, "Back-propagation, weight-elimination and time series prediction," in *Proc. 1990 Connectionist Models Summer School*, D. Touretzky, G. Hinton, and T. Sejnowski, Eds., Carnegie Mellon Univ., 1990, pp. 105-116.
- [60] A. S. Weigend, D. E. Rumelhart, and B. A. Huberman, "Generalization by weight-elimination with application to forecasting," in *Advances in Neural Information Processing Systems III*, D. S. Touretzky, Ed. San Mateo, CA: Morgan Kaufmann, 1991, pp. 875-882.
- [61] P. Werbos, "Beyond regression: New tools for prediction and analysis in the behavioral sciences," Ph.D. dissertation, Harvard Univ., Cambridge, MA, 1974.
- [62] —, "An overview of neural networks for control," *IEEE Contr. Syst. Mag.*, vol. 11, no. 1, pp. 40-41, Jan. 1991.
- [63] B. Widrow and D. Stearns, *Adaptive Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1985.
- [64] B. Widrow and R. Winter, "Neural nets for adaptive filtering and adaptive pattern recognition," *IEEE Computer*, pp. 25-39, Mar. 1988.
- [65] D. A. Wismer and R. Chattergy, *Introduction To Nonlinear Optimization: A Problem Solving Approach*. Amsterdam: North-Holland Publishing Company, 1978, ch. 6, pp. 139-162.
- [66] L. A. Zadeh, "Fuzzy sets," *Information and Control*, 8:338-353, 1965.
- [67] L. A. Zadeh, "Outline of a new approach to the analysis of complex systems and decision processes," *IEEE Trans. Syst., Man, Cybern.*, vol. 3, pp. 28-44, Jan. 1973.



Jyh-Shing Roger Jang was born in Taipei, Taiwan in 1962. He received the B.S. degree from National Taiwan University in 1984 and the Ph.D. degree from the University of California, Berkeley in 1992. He is currently a Research Engineer in the Department of Electrical Engineering and Computer Sciences at the University of California, Berkeley.

Since 1988, he has been a Research Assistant in the Electronics Research Laboratory at the University of California, Berkley. He spent the summer of 1991 and 1992 at the Lawrence Livermore National Laboratory, working on spectrum modeling and analysis using neural networks and fuzzy logic. His interests lie in the area of neurofuzzy modeling with applications to control, signal processing, and pattern classification.

Mr. Jang is a student member of American Association for Artificial Intelligence, and International Neural Networks Society.